
THE EVOLUTION OF MACHINE LEARNING AND AI

Week 4 Full Booklet

Optional reference · the required reading is the 20-page Core Booklet

Model Families: Structure Is an Assumption About the World

Award	Certificate in Artificial Intelligence and Machine Learning Engineering (30 ECTS)
Parent programme	Higher Diploma in Science in Computer Science (NFQ Level 8)
Module	The Evolution of Machine Learning and AI (5 ECTS, Semester 1)
Week	Week 4 of 12 · Sprint 1: Fast foundations and responsible AI up front
Indicative content	Items 4 and 5: convolutional and recurrent architectures; attention and transformer models
Learning outcomes	LO2: Characterise the training of the main families of model · LO3: Compare and contrast the main model architectures and their assumptions
Assessed in	Mid-module quiz, Week 6 (50% of module mark)
Companions	Convolution / Feature-Map Visualiser · Attention Visualiser · Sequence Memory Explorer · Lab: convolution, backpropagation through time and self-attention, all written out in numpy

Read this first. This is the **full** account of Week 4, kept for students who want to go deeper. **You are not required to read it.** The required reading is the short **Core Booklet**, and the Week 6 quiz can only ask what the Core Booklet's red boxes say. The **Reading Map** on the next page marks every chapter CORE (also covered, more briefly, in the Core Booklet) or EXTRA (optional). Boxes marked ◯ **Deeper Detail** go beyond what the lecture had time for. Boxes marked **Everyday Analogy** are the mental pictures the lecture used. Boxes marked **In Plain English** restate an idea with every piece of jargon removed; if a section loses you, skip to that box and come back. Boxes marked **Why This Matters** say what goes wrong if you do not understand the idea, and which later week depends on it. Boxes marked 📎 **Worked Example** are small enough to check with a pencil. New this week: ♦ **The Assumption** states an architecture's belief as a plain sentence about the world, **Measured in the Lab** attaches a real number to a claim, and ▶ **Open the Visualiser** points you at the interactive page that makes the idea manipulable.

Contents

1	Structure Is an Assumption	6
1.1	The move from “what shape” to “what connects”	6
1.2	Why the wiring is a claim about the world	7
1.3	The three questions to ask of any architecture	7
2	Inductive Bias	10
2.1	Why no assumptions means no learning	10
2.2	The exchange rate: assumptions for data	11
2.3	Where inductive bias actually lives	12
3	The Cost of Assuming Nothing	14
3.1	What a dense layer believes	14
3.2	The arithmetic of the parameter count	14
4	Weight Sharing	17
4.1	The three things sharing buys	17
4.2	What sharing does to the backward pass	17
5	Reading Architecture Notation	20
5.1	The shorthand, unpacked	20
5.2	The two shape questions	20
6	Convolution: The Operation	23
6.1	The operation	23
6.2	The eight lines of code	24
6.3	Padding and stride, and what each is for	25
7	What a Convolution Layer Assumes	27
7.1	Assumption one: locality	27
7.2	Assumption two: stationarity, also called translation equivariance	27
7.3	Assumption three: compositionality	29
8	Feature Maps, Channels, and What Filters Learn	31
8.1	One filter, one channel	31
8.2	Channels going in, channels coming out	31
8.3	What the layers actually learn	32
8.4	Activation functions, and why polarity matters	32
9	Pooling, Stride, and the Receptive Field	34
9.1	The receptive field	34
9.2	Pooling, and what it is really for	35
9.3	Equivariance against invariance	35
10	CNN Architectures, and What Each One Fixed	38
10.1	The lineage	38
10.2	ResNet, and the problem it actually solved	38
11	Where the CNN’s Assumptions Fail	41
11.1	The four ways locality and stationarity go wrong	41
12	Sequences: Why Order Is a Different Kind of Structure	43
12.1	The three things that change	43
12.2	What structure a sequence architecture could exploit	44
13	The Recurrent Network	46
13.1	The model	46
13.2	The code, in nine lines	46
14	Backpropagation Through Time, and the Vanishing Gradient Again	48
14.1	Backpropagation through time	48

14.2	The two standard patches, and their limits	49
15	LSTMs and GRUs: Gates as Learned Memory Control	51
15.1	The idea in one sentence	51
15.2	The equations, with the important line marked	51
15.3	GRUs, and the honest comparison	53
16	Where Recurrence Fails	55
16.1	Limit one: the fixed-size bottleneck	55
16.2	Limit two: strictly sequential computation	56
17	Attention: Relations as a First-Class Object	58
17.1	The soft dictionary	58
18	Self-Attention, Step by Step	61
18.1	The operation	61
18.2	Why three projections and not one	61
18.3	Why divide by the square root of the width	62
19	Multi-Head Attention, Position, and the Transformer Block	64
19.1	Multi-head attention	64
19.2	Attention cannot see order	64
19.3	Positional encoding	65
19.4	The transformer block	66
20	What the Transformer Assumes, and What It Gives Up	67
20.1	The bill	67
20.2	What is given up	68
21	The Scaling Bargain	69
21.1	The Vision Transformer, which is the cleanest evidence	69
22	The Same Three Families in PyTorch	72
22.1	The three families, in PyTorch	72
23	Choosing an Architecture	74
23.1	The decision table	74
23.2	The three questions	76
24	Structure, Assumption and Responsibility	78
24.1	The assumption you did not know you made	78
25	A Field Guide: Seven Questions for Any Architecture	80
26	The Revision Core: The Whole Week on Two Pages	81
26.1	The one sentence	81
26.2	The three families and their assumptions	81
26.3	The numbers to know	81
26.4	Convolution	82
26.5	Recurrence	82
26.6	LSTM	82
26.7	Attention	82
26.8	The bargain	82
26.9	Ten one-line lessons	83
27	Glossary in Plain English	84
28	Question Bank: Practising for Week 6	86
28.1	Short answer: two to three sentences each	86
28.2	Applied: read the numbers, do the arithmetic	87
28.3	Discussion prompts	88
29	Further Reading	89
	Appendix: How Week 4 Maps to the Module	90

Reading map: what is core and what is extra

CORE means the idea is examinable and the Core Booklet already covers it in plain English; read the chapter here only for the fuller version. **EXTRA** means optional: interesting, not examined, safe to skip.

Ch.	Title	Status	Time
1	Structure Is an Assumption	CORE	10 min
2	Inductive Bias	CORE	10 min
3	The Cost of Assuming Nothing	CORE	8 min
4	Weight Sharing	CORE	8 min
5	Reading Architecture Notation	EXTRA	8 min
6	Convolution: The Operation	CORE	12 min
7	What a Convolution Layer Assumes	CORE	10 min
8	Feature Maps, Channels, and What Filters Learn	EXTRA	8 min
9	Pooling, Stride, and the Receptive Field	CORE	10 min
10	CNN Architectures, and What Each One Fixed	EXTRA	8 min
11	Where the CNN's Assumptions Fail	CORE	6 min
12	Sequences: Why Order Is a Different Kind of Structure	CORE	8 min
13	The Recurrent Network	CORE	8 min
14	Backpropagation Through Time, and the Vanishing Gradient Again	CORE	12 min
15	LSTMs and GRUs: Gates as Learned Memory Control	CORE	12 min
16	Where Recurrence Fails	CORE	8 min
17	Attention: Relations as a First-Class Object	CORE	6 min
18	Self-Attention, Step by Step	CORE	12 min
19	Multi-Head Attention, Position, and the Transformer Block	CORE	10 min
20	What the Transformer Assumes, and What It Gives Up	CORE	6 min
21	The Scaling Bargain	CORE	8 min
22	The Same Three Families in PyTorch	EXTRA	8 min
23	Choosing an Architecture	EXTRA	5 min
24	Structure, Assumption and Responsibility	EXTRA	5 min
25	A Field Guide: Seven Questions for Any Architecture	CORE	3 min
26	The Revision Core: The Whole Week on Two Pages	CORE	8 min
27	Glossary in Plain English	CORE	6 min
28	Question Bank: Practising for Week 6	EXTRA	40 min
29	Further Reading	EXTRA	2 min

Inside a CORE chapter, the boxes marked  Deeper Detail and  Worked Example are themselves extra. The examinable statements are exactly the red ★ boxes in the Core Booklet; nothing here adds to that list.

1 Structure Is an Assumption

COVERS	The one sentence the whole week is built on, and what follows from it
TIME	Read first, about 20 minutes
IN BRIEF	Week 3 said that choosing a model is choosing an assumption. Week 4 says which assumptions, made how, and what each one is worth in data you did not have to collect.

Week 3 ended on a claim that was easy to nod at and hard to feel: **choosing a model is choosing an assumption, and when the assumption matches reality you need less data.** In Week 3 that claim stayed small. A straight line, a set of right-angled splits, a widest corridor. Four shapes of boundary on a two-dimensional plot.

This week the claim gets large, and it stops being about the shape of a boundary. It becomes about **which parts of the input are allowed to talk to each other at all.**

1.1 The move from “what shape” to “what connects”

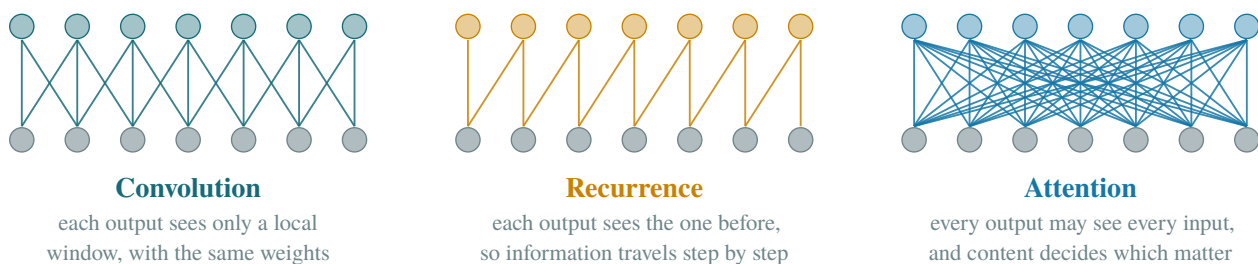
A neural network is a set of numbers and a wiring diagram. Week 3 spent its time on the numbers: how they are adjusted, how blame is assigned, how fast to step. It treated the wiring as fixed, because in Week 3 it was. Every network you built connected everything to everything.

This week the wiring becomes the variable, and the numbers become the boring part. The training loop does not change. Forward, loss, backward, update, exactly as before. What changes is the first box, and specifically **which connections exist inside it.**

KEY IDEA

Every architecture in this week is a decision about **which inputs may influence which outputs, and which weights are shared between them.** That decision is not a technical detail or an efficiency measure. It is a claim about how the data is structured, and it is true or false about your particular data.

When the claim is true, you get a model that needs less data, generalises further, and often runs faster. When the claim is false, you get a model that is worse than the one that assumed nothing.



Look at that figure for longer than feels necessary, because the rest of the week is commentary on it.

Three panels, seven inputs, seven outputs. The only difference between them is which lines exist. In the first, each output is wired to a small neighbourhood of inputs and every output uses the same weights. In the second, each output is wired to the input beneath it and to the output before it, so information travels along a chain. In the third, every output is wired to every input, and how strongly is decided by the content of the inputs themselves.

Those three pictures are a convolutional network, a recurrent network, and an attention layer. Everything else is bookkeeping.

IN PLAIN ENGLISH

Imagine three ways of organising a large office to answer a question.

The first office seats people in a grid and lets each person talk only to their immediate neighbours. To get information across the room it has to be passed along, desk by desk, being summarised at each step. Everybody follows the same conversation rules, so you only have to train one person and copy them. This is a convolution.

The second office is a single long corridor of desks. Each person receives a note from the desk before them, adds what they know, and passes it on. The person at the end answers the question. Everybody uses the same procedure for updating the note. This is recurrence.

The third office is one big room where anybody may talk to anybody, and before each round everyone shouts out what they have and what they need, so the right pairs find each other directly. Nobody has to relay anything. This is attention.

None of these is the right way to run an office. Which one works depends entirely on what the question is and where the information sits.

1.2 Why the wiring is a claim about the world

Here is the part that is genuinely worth slowing down for. A restriction on wiring sounds like a limitation, something you accept reluctantly to save memory. It is much more interesting than that.

Suppose you wire a layer so that each output depends only on nearby inputs. You have not merely saved parameters. You have asserted something: **that in this data, things which are close together are related, and things which are far apart are not directly related.** That sentence is either true or false about your data, and it is checkable.

For a photograph it is close to true. A pixel's meaning is set by its neighbours; the pixel in the opposite corner is almost irrelevant to whether this pixel is part of an edge. For a spreadsheet of customer records it is nonsense, because column 3 and column 47 are neighbours only in the sense that somebody typed them in that order.

◆ THE ASSUMPTION: LOCALITY

The plain sentence: what something means is decided by what is near it.

Where it is true: images, audio spectrograms, video, protein sequences, anything laid out in a space where nearness is meaningful.

Where it is false: tabular data with arbitrary column order, sets of unordered items, graphs where the useful connections are not spatial.

What it buys when true: a very large reduction in parameters, and the ability to learn a feature once instead of once per position.

What it costs when false: the model is forced to route real long-range relationships through many layers of local summarising, and it may never find them.

1.3 The three questions to ask of any architecture

For the rest of the module, whenever you meet a new architecture, ask three things in this order. They are the operational version of this chapter.

- 1. What is connected to what?** Which inputs can influence which outputs, directly, in one layer?
- 2. Which weights are shared?** Is the same set of numbers reused in different places, and if so, across what: space, time, or nothing?

3. What does that combination assume about the data, stated as a sentence a non-specialist would understand?

If you can answer those three for a paper's architecture diagram, you can predict most of the paper's results before reading them: what data regime it will win in, what it will fail at, and what its authors will have had to patch.

WHY THIS MATTERS

This is the difference between memorising architectures and understanding them, and it is worth a great deal beyond the exam.

There are perhaps two hundred named architectures you might meet in a career, and new ones arrive monthly. Nobody remembers two hundred things. What people who are good at this actually carry around is a short list of **assumptions**, plus the habit of asking which ones a given diagram encodes. A “ConvNeXt” or a “Mamba” or a “Perceiver” then becomes a specific combination of familiar assumptions rather than a new thing to learn from scratch.

It also changes what you do when a model underperforms. The instinct is to get more data or make the model bigger. The question this chapter gives you is different and usually cheaper: **is my architecture's assumption actually true about my data?** That question has been the answer to more real project failures than any hyperparameter.

COMMON MISTAKE

“A more flexible model is a better model, and restrictions are always a compromise.”

This is the single most costly wrong belief in the week, and it feels obviously right, which is why it survives.

The dense network in this week's lab is **strictly more expressive** than the convolutional one. Every function the CNN can compute, the MLP can also compute; the reverse is not true. The MLP has 24 times as many parameters. On the lab's task, the CNN scores 0.945 from 128 training images and the MLP scores 0.275, which is barely above the 0.250 you get by guessing.

More expressive lost, and lost badly. Flexibility is not capability. A model that can represent every possible answer has no reason to prefer the right one, so it needs enough data to rule out all the wrong ones, and there are astronomically many wrong ones.

MEASURED IN THE LAB

The whole of this chapter, in four numbers from `inductive_bias.py`:

Setting	CNN, 700 parameters	MLP, 16,708 parameters
128 training images	0.945	0.275
512 training images	0.992	0.328
positions never seen in training	0.998	0.253
pixel order shuffled	0.255	0.333

Read the last row against the others. Shuffle the pixels, which destroys adjacency and nothing else, and the smaller model's advantage does not shrink; it **inverts**. Its whole advantage was the assumption.

QUIZ WATCH · WEEK 6

Near certain in some form: state what an architecture encodes, in one sentence. Explain why a more expressive model can perform worse than a less expressive one on the same data. Given a data type (photographs, sensor traces, customer records, sentences), say which structural assumption is appropriate and why.

2 Inductive Bias

COVERS	The proper name for the thing Chapter 1 described, and why every useful model must have one
TIME	About 18 minutes; the vocabulary chapter for the week
IN BRIEF	Inductive bias is everything a model believes before it sees any data. It is not a flaw to be minimised. Without it, learning from finite data is impossible in principle.

The idea in Chapter 1 has a name, and it is an unfortunate one. **Inductive bias** sounds like a defect, especially after Week 2 spent a fortnight on bias in the sense of unfairness. These are different senses of the word and it is worth separating them cleanly before going further.

- **Bias** in Week 2's sense: a systematic disadvantage imposed on people, an ethical and legal problem.
- **Bias** as a parameter, from Week 3, Chapter 4: the constant b added at the end of a weighted sum.
- **Inductive bias**, this week: the set of preferences a learning algorithm has about which answers to favour, before seeing any data at all.

The third sense is the one that matters here, and in that sense bias is not merely acceptable. It is compulsory.

2.1 Why no assumptions means no learning

Consider a genuinely small problem. You are shown three examples: the inputs 1, 2 and 3 map to the outputs 2, 4 and 6. What does 4 map to?

Almost everyone answers 8. Now justify it. Nothing in those three examples rules out the rule “double the input, unless it is 4, in which case return 1,000”. Nothing rules out “double the input for inputs below 4, then return the input squared”. There are infinitely many rules agreeing with every example you have seen and disagreeing about the next one.

You answered 8 because you have a **preference** for simple rules over complicated ones, applied before you saw the data. That preference is your inductive bias. It is not derived from the examples; it is what let you use the examples at all.

KEY IDEA

Any finite set of examples is consistent with infinitely many rules. Choosing between those rules requires a preference that does not come from the data. That preference is the inductive bias, and a learner without one cannot generalise, not because it is badly built, but because generalisation without prior preference is not a well-defined operation.

So the question is never **whether** your model has assumptions. It is **which** assumptions, whether you chose them deliberately, and whether they are true of your problem.

IN PLAIN ENGLISH

Inductive bias is **what the model believes before it has seen anything**, and every model has beliefs whether or not its author noticed.

Think of hiring somebody to guess the rules of a game by watching a few rounds. A candidate who says “I assume the rules are consistent and reasonably simple” will learn fast if that is true of the game. A candidate who insists on treating every possible rule as equally likely, including “the rules change on Thursdays”, will need to watch for years, because they cannot rule anything out.

The second candidate is not being more scientific. They are being unemployable.

◉ DEEPER DETAIL: No free lunch, and what it actually says

There is a formal result here, usually cited and rarely stated correctly. Wolpert's **no free lunch** theorems say roughly this: averaged over **all possible problems**, every learning algorithm performs identically, and no better than random guessing.

People take this to mean that model choice does not matter. It means precisely the opposite.

The average is over all possible problems, which includes an overwhelming majority of problems in which the data is structureless noise with an arbitrary labelling. Those problems do not occur in practice. The universe generates images with local structure, language with syntax, and physical processes with continuity, and it does not generate the pathological majority.

So the correct reading is: **there is no universally best model, therefore performance comes entirely from matching your assumptions to the actual structure of your actual problem.** No free lunch is not an argument against inductive bias; it is the reason inductive bias is the whole game.

2.2 The exchange rate: assumptions for data

Here is the practical relationship, and it is close to a law.

A correct assumption is worth data. Every constraint you build into an architecture is a set of possible answers the model no longer has to rule out by example. If the constraint is true, you get the same accuracy from fewer examples. If you have no constraint, you must pay for the same conclusion in data.

An incorrect assumption is worth negative data. A model prevented from representing the truth cannot be rescued by more examples. It will converge confidently to the best available wrong answer.

MEASURED IN THE LAB

The exchange rate, measured. `inductive_bias.py` runs both models on the same task with increasing amounts of data:

Training images	CNN test	MLP test	MLP training accuracy
64	0.723	0.252	1.000
128	0.945	0.275	1.000
256	0.997	0.252	1.000
512	0.992	0.328	1.000

Then the MLP alone, given four times the capacity (66,820 parameters) and far more data:

Training images	MLP test	Verdict
1,024	0.390	still poor
4,096	0.730	getting there
16,384	0.948	finally matches the CNN's score at 128 images

The exchange rate on this task is about 128 to 1. The CNN reached 0.945 from 128 images with 700 parameters. The MLP needed roughly 16,384 images and 95 times the parameters to match it.

Now look at the third column of the first table. The MLP's **training** accuracy is 1.000 at every size, including 64 images. It is not failing to learn. It is learning the training set perfectly and learning nothing that transfers, which is Week 3's overfitting arriving with a new cause: not too much capacity in the abstract, but capacity pointed in no particular direction.

2.3 Where inductive bias actually lives

It is not only in the architecture, and this is worth knowing because it tells you where else to look when something is wrong. Assumptions enter a system in at least five places.

Where	The assumption	Example
Architecture	which inputs may interact, and which weights are shared	a convolution assumes locality and stationarity
Loss function	which mistakes matter and by how much	squared error assumes large errors are disproportionately bad
Regularisation	which solutions are preferable among equally accurate ones	weight decay assumes smaller weights are more likely correct
Preprocessing	what is signal and what is noise	centring digits assumes position carries no information
Data augmentation	which transformations should not change the answer	flipping images assumes left and right are equivalent

That fourth row is quietly important, and the lab measures it.

MEASURED IN THE LAB

`experiment_no_payoff()` runs the same two models on small, pre-centred, pre-cropped handwritten digits.

Model	Parameters	Test accuracy
CNN, global pooling	754	0.940
MLP	4,810	0.978

The MLP wins. This is not a failure of Week 4's thesis; it is the thesis applied honestly.

Somebody centred those digits before you got them. Centring is a hand-built solution to exactly the problem that translation equivariance solves, applied in the preprocessing step instead of the architecture. The variation the convolution was designed to handle has already been removed, and **you do not get paid twice for solving a problem once**.

Report results like this one. A student who can explain why the CNN lost here understands the week considerably better than one who only has the winning numbers.

WHY THIS MATTERS

The exchange rate between assumptions and data is the single most useful idea in this module for anybody who will work with limited data, which is almost everybody outside a large laboratory.

Frontier language models are trained on a substantial fraction of the public internet. You will not be. You will have four thousand labelled examples from one hospital, or six months of readings from one factory. In that regime you cannot buy your way out with data, so the only lever you have is **choosing structure that already knows something true about your problem**.

This is exactly why Week 5 follows Week 4. Week 5 asks the natural next question: if a convolution is worth 128 times the data because it knows that neighbouring pixels are related, what is it worth to build in something you know far more precisely, such as conservation of energy? Chapter 21 returns to this.

► **OPEN THE CONVOLUTION / FEATURE-MAP VISUALISER**

Open [interactive/convolution-feature-map-visualiser.html](https://interactive.convolution-feature-map-visualiser.html) and go to tab 4, “Break the assumption”. Choose the hand-drawn 7 and switch between **shift it by 5 pixels** and **shuffle the pixels**.

Watch the edge-detector response. Under the shift it moves and survives. Under the shuffle it collapses, even though the histogram panel on the right shows every pixel value is still present and unchanged. That contrast is inductive bias made visible in about ten seconds, and it is the same experiment the lab runs on a trained network.

QUIZ WATCH · WEEK 6

Highly likely: define inductive bias in one sentence and explain why a learner without one cannot generalise. State the relationship between assumptions and data requirements. Name three places other than the architecture where assumptions enter a system. A common applied item gives you the “MLP wins on centred digits” result and asks you to explain it without abandoning the week’s thesis.

3 The Cost of Assuming Nothing

COVERS	What a fully connected layer actually commits to, and the arithmetic of why it struggles on structured data
TIME	About 15 minutes
IN BRIEF	The dense layer is the honest baseline: it assumes nothing about the input's structure. This chapter counts precisely what that costs.

Before looking at architectures that assume something, it is worth being exact about the one that does not. This is the network from Week 3, Chapter 14: every input connected to every unit in the next layer.

3.1 What a dense layer believes

A fully connected layer takes its input as a flat list of numbers and computes a weighted sum for each unit. There is one weight per input per unit, and none of them are tied together.

So what does it assume? Almost nothing, and specifically it does not assume that the order of the inputs means anything. To a dense layer, the input is a **bag of numbered slots**. Slot 1 and slot 2 are no more related than slot 1 and slot 700.

IN PLAIN ENGLISH

A dense layer treats a photograph the way a spreadsheet treats a row of figures: as a list of values in a fixed order, where the order is just an arbitrary convention for keeping track of things.

If you took every photograph in your dataset and applied the same scrambling to the pixels, a dense layer would be almost entirely unaffected, because it never knew which pixel was next to which in the first place. That sounds like an odd claim. It is measured in this week's lab, and it is exactly what happens.

MEASURED IN THE LAB

The pixel shuffle, from `experiment_shuffle()`. One fixed random permutation of the 256 pixels, applied to every image in training and testing:

Pixel order	CNN	MLP	Gap
original	0.992	0.328	+0.664
shuffled	0.255	0.333	−0.078

The MLP moved by half a percentage point, and slightly **upward**, which is noise. It genuinely did not notice, because it never used adjacency.

The CNN fell from 0.992 to 0.255, which on a four-class task is chance. Not degraded: destroyed.

No information was removed. The permutation is a bijection, exactly as reversible as a substitution cipher, and the label remains perfectly determined by the image. What was removed is the **meaning** of adjacency, and adjacency is the only thing the convolution ever assumed.

3.2 The arithmetic of the parameter count

The other cost is countable, and the numbers are more dramatic than most people expect.

MEASURED IN THE LAB

From `convolution.py`, comparing one dense layer of 128 units against one convolution layer of 32 filters, on the same input:

Input	Pixels	Dense, 128 units	Conv, 32 filters of 3×3	Ratio
28 × 28	784	100,480	320	314 to 1
64 × 64	4,096	524,416	320	1,639 to 1
224 × 224	50,176	6,422,656	320	20,071 to 1

Note the third column growing and the fourth column standing still. **A convolution layer’s parameter count does not depend on the size of the image at all.** One 224 × 224 colour photograph fed to one dense layer of 128 units needs over 19 million weights in the first layer alone.

WORKED EXAMPLE: Why the convolution column does not move

A dense layer needs one weight for every (input, output) pair. The number of inputs is the number of pixels, so the count scales with the image area.

A convolution layer needs one weight for every (position in the kernel, input channel, output channel) triple. The kernel is 3 × 3 regardless of the image. So:

$$\text{params} = 32 \text{ filters} \times (3 \times 3 \times 1 \text{ channel}) + 32 \text{ biases} = 288 + 32 = 320$$

Nothing in that expression mentions the image size. Change 28 × 28 to 4,000 × 4,000 and it is still 320.

This is worth pausing on, because it has a consequence beyond memory: **the same trained convolution layer can be applied to an image of any size.** A dense layer trained on 28 × 28 inputs is undefined on a 29 × 29 input. Chapter 9 shows how to carry that property all the way to the output.

○ DEEPER DETAIL: The dense layer is not useless, and where it belongs

None of this makes dense layers obsolete, and it would be a mistake to leave this chapter thinking so.

Dense layers are correct when the input genuinely has no exploitable structure. Customer records, laboratory measurements, engineered features: there is no meaningful sense in which “age” is adjacent to “postcode”, and a convolution over those columns would be assuming something false.

Dense layers appear inside almost every modern architecture. The feed-forward block inside a transformer, which is roughly two thirds of its parameters, is a pair of dense layers applied to each position. What changed is not that dense layers went away; it is that they stopped being asked to discover structure that could be built in instead.

The right mental model: use structure where you have it, and dense layers where you do not.

COMMON MISTAKE

“The MLP failed because it did not have enough parameters or training.”

It had 24 times as many parameters as the CNN, and in the scaling experiment 95 times as many. It reached 1.000 training accuracy in every single run, at every dataset size, including 64 images. It was trained with the same optimiser for the same number of epochs.

It was not starved of capacity or of training. It was starved of **reasons to prefer one of the many rules that fit its training data.** That is what an inductive bias supplies, and it had none.

WHY THIS MATTERS

Being able to count parameters and say what a layer assumes is a practical skill you will use immediately.

It tells you which architecture can even be trained on the hardware and data you have. It tells you where a model is likely to overfit, because the layers with the most parameters and the weakest assumptions are the usual suspects. And it gives you the correct reply to the most common bad suggestion in applied machine learning, which is “let us just make it bigger”. Sometimes the right answer is to make it smaller and better shaped.

QUIZ WATCH · WEEK 6

Likely: given an input size and layer specification, compute the parameter counts for a dense and a convolutional layer and state the ratio. Explain why the convolution’s count is independent of input size, and give one consequence. Explain what the pixel-shuffle experiment demonstrates and why it is a fair test.

4 Weight Sharing

COVERS	The single mechanism underneath convolution and recurrence alike, and what it does to the gradient
TIME	About 15 minutes; the most transferable idea in the week
IN BRIEF	Convolution shares weights across space. Recurrence shares them across time. It is one idea, and understanding it once explains both families.

Two of the three families this week rest on the same trick, and seeing it as one trick rather than two is a considerable saving.

Weight sharing means using the same numbers in more than one place, deliberately, and forcing them to stay equal during training.

4.1 The three things sharing buys

1. **Fewer parameters.** Obvious, and the least interesting of the three.
2. **A feature learned once works everywhere.** This is the important one. If a detector for a horizontal edge is learned from examples in the top left of the image, weight sharing means it **already** works in the bottom right, because it is literally the same detector.
3. **Independence from input size.** A shared kernel or a shared recurrent matrix can be applied any number of times, so the model accepts inputs larger or longer than anything it trained on.

IN PLAIN ENGLISH

Weight sharing is **training one inspector and cloning them**, instead of training a separate inspector for each position on the production line.

If a defect looks the same wherever it appears, cloning is obviously right: you get many more training examples per inspector, because every position's data teaches the same one person. And when a defect appears at a position that never had one during training, your clone already knows what to do.

If defects genuinely look different at different positions, cloning is wrong, and you have forced every position to accept one compromise inspector.

4.2 What sharing does to the backward pass

This is the part that most treatments skip, and it is where the idea becomes concrete rather than decorative. Recall Week 3, Chapter 14: backpropagation computes, for each weight, how much the loss would change if that weight moved.

Now ask what that means for a **shared** weight. A kernel weight is used at hundreds of positions in one image. Each of those uses contributes its own effect on the loss. Since the weight must remain a single number, its gradient is the **sum** of all those contributions.

WORKED EXAMPLE: Weight sharing as one line of code

From `cnn_numpy.py`, inside `Conv2D.backward`:

```
d_flat = d_out.transpose(0, 2, 3, 1).reshape(-1, d_out.shape[1])
self.dW = (d_flat.T @ self.cols).reshape(self.W.shape)
```

`d_flat` has one row for every (image, position) pair in the batch. The matrix multiply sums over **all** of them. So if a batch of 16 images each has 256 positions, that single line adds up 4,096 separate contributions into the same $3 \times 3 \times \text{channels}$ block of numbers.

Compare a dense layer, where each weight is used exactly once per example and receives exactly one contribution.

There is a real consequence here, and it is one of the underrated reasons convolutional networks are so data-efficient: **one image supplies hundreds of gradient contributions to the same small set of weights.** A dense layer given the same image supplies one contribution to each of a much larger set. The convolutional layer extracts far more learning signal per training example, which is the mechanism behind the 128-to-1 exchange rate in Chapter 2.

KEY IDEA

Weight sharing is the same bargain everywhere it appears: **a large reduction in the number of things to learn, and a large increase in the amount of evidence per thing learned, in exchange for committing to an assumption about where the same rule applies.**

A convolution commits to: the same rule applies at every position in space.

A recurrent network commits to: the same rule applies at every step in time.

Attention, notably, shares its projection weights across positions too, which is why it also handles variable-length input. What it does **not** share or restrict is which positions may interact.

◉ DEEPER DETAIL: Sharing against tying against grouping

Three related mechanisms worth distinguishing, because papers use them casually.

Weight sharing proper: the same weights used at many positions, as in a convolution. The positions are different, the weights identical.

Weight tying: two structurally different layers forced to use the same matrix, most famously a language model's input embedding and output projection. This assumes that the representation of "the word cat as input" and "the word cat as a prediction" should be the same vector, which saves a large number of parameters in a model with a big vocabulary.

Grouped or depthwise convolution: the channels are split into groups, and sharing happens within groups but not across them. This assumes some channels are related and others are not, and it is the trick behind the efficient architectures used on phones. MobileNet's depthwise separable convolution reduces the cost of a layer by roughly the number of channels, at the price of assuming that spatial mixing and channel mixing can be done separately.

› OPEN THE SEQUENCE MEMORY EXPLORER

Open [interactive/sequence-memory-explorer.html](https://interactive.sequence-memory-explorer.html), tab 3, "Unrolling, and weight sharing".

Drag the sequence length slider. Watch the parameter table: the RNN and LSTM counts do not move at all, while the dense network's count grows linearly and the note points out that the dense model can only ever accept sequences of exactly that one length.

Then read the amber caption under the diagram. Every W in that unrolled picture is the **same** matrix, which is the single most commonly misread thing about recurrent networks.

COMMON MISTAKE

"An unrolled RNN diagram shows a deep network with many layers."

It shows **one** layer applied many times. The boxes are copies of the same weights at different moments, not different weights. This matters enormously for Chapter 14: because it is the same matrix at every step, the gradient becomes a power of one matrix rather than a product of unrelated ones, and that is a much worse numerical situation.

WHY THIS MATTERS

Weight sharing is the idea that generalises furthest beyond this module.

Graph neural networks share weights across nodes, assuming the same update rule applies to every node. Set-based architectures share across elements, assuming order carries nothing. Equivariant networks used in molecular modelling share across rotations, assuming physics does not care which way up your coordinate system is. Every one of those is this chapter with a different symmetry substituted in.

Once you see architecture design as **choosing which transformations should leave the answer unchanged, and then sharing weights across them**, the field's apparent variety collapses into something you can reason about.

QUIZ WATCH · WEEK 6

Likely: state the three benefits of weight sharing and the assumption each requires. Explain what happens to the gradient of a shared weight and why. Explain why an unrolled RNN is not a deep network in the ordinary sense, and one consequence.

5 Reading Architecture Notation

COVERS	A decoder for the shorthand in every architecture diagram and paper you will read
TIME	About 10 minutes; return here whenever a specification stops you
IN BRIEF	Architecture descriptions are compressed to the point of looking cryptic. The compression is regular and this chapter unpacks it.

Week 3, Chapter 2 decoded mathematical notation. This chapter does the same for architecture specifications, which are their own dialect and are usually explained nowhere.

5.1 The shorthand, unpacked

You will see	It means	In plain English
<code>Conv2d(3, 64, 7, stride=2, padding=3)</code>	a 2-D convolution, 3 input channels, 64 output channels, 7×7 kernel, stride 2, padding 3	take a colour image, apply 64 different 7×7 detectors, and produce a half-size output with 64 channels
<code>3x3, 64</code>	a 3×3 kernel producing 64 channels	64 small detectors; the input channel count is implied by the previous layer
$H \times W \times C$	height, width, channels	the shape of a feature map: a stack of C images of size $H \times W$
<code>/2</code>	stride 2, or downsampling by 2	halve the resolution, deliberately
<code>x4</code> or <code>x4</code>	this block repeated four times	four copies in series, each with its own weights
<code>d_model = 512</code>	the width of a transformer's representation	every token is described by 512 numbers throughout the model
<code>n_heads = 8</code>	eight attention heads	the 512 numbers are cut into 8 slices of 64, each attending separately
<code>GAP</code>	global average pooling	average each channel over all positions, giving one number per channel
<code>BN, LN</code>	batch or layer normalisation	rescale the activations so training is stable; Chapter 10
T, L, n	sequence length	how many tokens, steps or positions there are

5.2 The two shape questions

For any layer, two questions answer nearly everything, and you should be able to do both on paper.

What shape comes out? For a convolution, the size arithmetic is one formula worth memorising:

$$\text{out} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$$

where n is the input size, k the kernel size, p the padding and s the stride.

How many parameters? For a convolution:

$$\text{params} = C_{\text{out}} \times (C_{\text{in}} \times k \times k) + C_{\text{out}}$$

WORKED EXAMPLE: Reading one real specification

Here is the first layer of ResNet-50, written as it usually appears:

conv1: 7x7, 64, stride 2 on a $224 \times 224 \times 3$ input.

Shape out. Padding is 3 (chosen so the arithmetic comes out even):

$$\left\lfloor \frac{224 + 6 - 7}{2} \right\rfloor + 1 = \left\lfloor \frac{223}{2} \right\rfloor + 1 = 111 + 1 = 112$$

So the output is $112 \times 112 \times 64$.

Parameters. $64 \times (3 \times 7 \times 7) + 64 = 64 \times 147 + 64 = 9,472$.

Sanity check. Nine and a half thousand parameters, to process a 150,000-number input into a 800,000-number feature map. A dense layer doing anything comparable would need billions. The whole of ResNet-50 is about 25 million parameters, which is fewer than one dense layer of 512 units on this input would need.

`convolution.py` prints spot checks for this formula. The value for $n = 224, k = 7, s = 2, p = 3$ is 112, which agrees.

IN PLAIN ENGLISH

Padding is **drawing a border of zeros around the image before you start**, so the kernel can be centred on the edge pixels. Without it, every convolution shrinks the image slightly and edge pixels get less attention than middle ones. Padding of $\frac{k-1}{2}$ keeps the size exactly the same, which is why you see padding=1 with 3x3 almost universally.

Stride is **how far the kernel jumps between positions**. Stride 1 looks everywhere; stride 2 looks at every other position, producing an output half the size. It is a deliberate discarding of resolution in exchange for a wider view and less computation.

TRY IT YOURSELF

Do these four on paper, then check with `output_size()` in `convolution.py`.

1. $n = 32, k = 3, s = 1, p = 1$. (Answer: 32. This is the “keep the size” setting.)
2. $n = 32, k = 3, s = 1, p = 0$. (Answer: 30. Two pixels lost.)
3. $n = 32, k = 5, s = 2, p = 2$. (Answer: 16.)
4. A `Conv2d(64, 128, 3, padding=1)` layer: how many parameters? (Answer: $128 \times 64 \times 9 + 128 = 73,856$.)

WHY THIS MATTERS

This is a small chapter with a disproportionate payoff, because it is the difference between reading a paper’s architecture table and skipping it.

Every architecture paper contains a table of layer specifications, and that table is the actual content: the prose around it is mostly motivation. Being able to read it means you can tell whether a model will fit in your memory budget, where its parameters actually are, and whether the authors' claimed efficiency comes from a genuine idea or from making something smaller.

QUIZ WATCH · WEEK 6

The size formula is an extremely likely applied item, in the form of a specification with numbers to compute. Practise until you can do it without hesitating, in both directions: given input and output sizes, what stride was used?

6 Convolution: The Operation

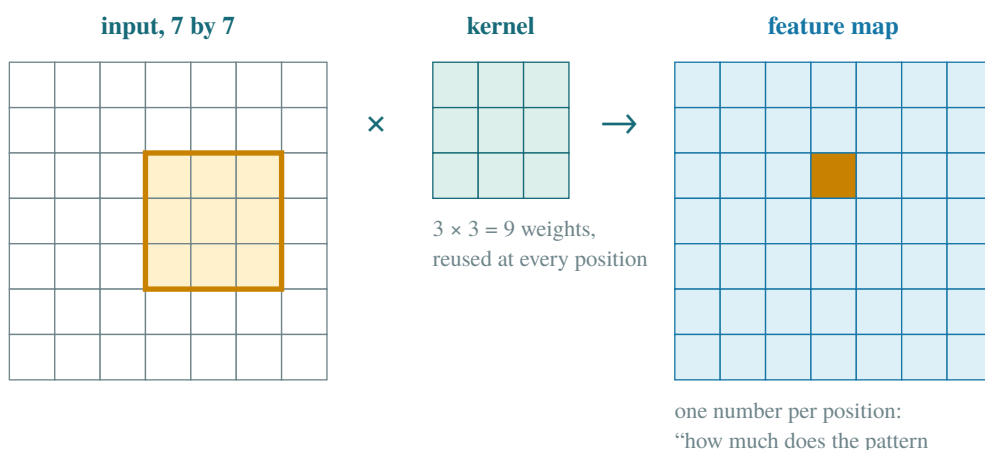
COVERS	What a convolution actually computes, in arithmetic simple enough to do by hand
TIME	About 18 minutes; do the worked example with a pencil
IN BRIEF	A convolution is a weighted sum of a small patch, repeated at every position with the same weights. That is the entire operation, and everything else in the chapter follows from it.

The word “convolution” does more to obscure this operation than any other piece of vocabulary in the module. The operation itself is something you could explain to a child.

6.1 The operation

Take a small grid of numbers, usually 3×3 . Lay it over the top left corner of your image. Multiply each of the nine numbers by the pixel underneath it, add up the nine products, and write the answer down. Slide one pixel right and do it again. When you reach the end of the row, drop down and start the next.

That is it. Nine multiplications and eight additions per position, with the **same nine numbers** at every position.



IN PLAIN ENGLISH

A convolution is **one small question, asked of every location in turn**.

The nine numbers are the question. A question like “is there a vertical edge here?” can be written as nine numbers: negative on the left, zero in the middle, positive on the right. Lay that over a patch that is dark on the left and bright on the right, and the sum comes out large. Lay it over a uniform patch and the sum comes out near zero, because the negatives and positives cancel.

The output is a map of the answers: one number per location, saying how strongly the question was answered “yes” right there. That is why it is called a **feature map**. It is a map of where a feature is.

WORKED EXAMPLE: A 3×3 convolution, done entirely by hand

Here is a patch of an image, with values between 0 and 1, showing a vertical edge (dark left, bright right):

$$\begin{pmatrix} 0.1 & 0.2 & 0.9 \\ 0.1 & 0.2 & 0.9 \\ 0.1 & 0.2 & 0.9 \end{pmatrix}$$

And here is a vertical-edge kernel (the Sobel operator, one of the classics):

$$\begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

Multiply position by position and add:

$$(0.1)(-1) + (0.2)(0) + (0.9)(1) = -0.1 + 0 + 0.9 = 0.8$$

$$(0.1)(-2) + (0.2)(0) + (0.9)(2) = -0.2 + 0 + 1.8 = 1.6$$

$$(0.1)(-1) + (0.2)(0) + (0.9)(1) = -0.1 + 0 + 0.9 = 0.8$$

Total: $0.8 + 1.6 + 0.8 = 3.2$. A large positive response, correctly reporting a strong dark-to-bright vertical edge.

Now try the same kernel on a **uniform** patch where every value is 0.5:

$$(0.5)(-1 + 0 + 1) + (0.5)(-2 + 0 + 2) + (0.5)(-1 + 0 + 1) = 0$$

Zero. The kernel's weights sum to zero, so any flat region gives no response at all. This is not a coincidence: it is how you design a detector that responds to **change** rather than to brightness.

Finally, try it on a patch that is bright left, dark right, the opposite edge:

$$\begin{pmatrix} 0.9 & 0.2 & 0.1 \\ 0.9 & 0.2 & 0.1 \\ 0.9 & 0.2 & 0.1 \end{pmatrix}$$

By symmetry the answer is -3.2 . The **sign** carries the direction of the edge. This is why Chapter 8's ReLU matters: it keeps one polarity and discards the other, so that "edge this way round" becomes a distinct feature from "edge the other way round".

6.2 The eight lines of code

The whole operation, from `convolution.py`:

```
def convolve2d(image, kernel, stride=1, padding=0):
    if padding:
        image = np.pad(image, padding, mode="constant")
    kh, kw = kernel.shape
    h = (image.shape[0] - kh) // stride + 1
    w = (image.shape[1] - kw) // stride + 1
    out = np.zeros((h, w))
    for i in range(h):
        for j in range(w):
            patch = image[i*stride:i*stride+kh, j*stride:j*stride+kw]
            out[i, j] = np.sum(patch * kernel)
    return out
```

Note what is **absent** from that code. There is no lookup of a different weight per position. `kernel` is the same object on every iteration of both loops. That absence is the stationarity assumption, expressed as code rather than as a sentence, and Chapter 7 is about what it commits you to.

◉ DEEPER DETAIL: Why this is technically a cross-correlation, and why nobody cares

A mathematician's convolution flips the kernel before sliding it: the formula involves $f(t - \tau)$ rather than $f(t + \tau)$. What every deep learning library computes, and what the code above computes, is the **unflipped** version, properly called a cross-correlation.

The distinction is genuinely irrelevant here, and it is worth knowing why rather than just being told. In classical signal processing the kernel is **given**, so a flip changes your answer. In deep learning the kernel is **learned**. If the flipped version were the better detector, gradient descent would simply learn the flipped weights. The set of functions the layer can represent is identical either way.

So the field kept the more familiar word for the slightly wrong operation, and nothing has ever gone wrong as a result. Do know the distinction, because examiners occasionally ask and because it explains why signal-processing textbooks look different.

6.3 Padding and stride, and what each is for

Padding adds a border of zeros before the sliding starts. Two reasons. First, without it every convolution shrinks the output, and a twenty-layer network with 3×3 kernels would shrink the image by 40 pixels, which is most of a small image. Second, without padding the kernel can never be centred on an edge pixel, so edge pixels take part in fewer sums and get systematically less influence.

Stride is how far the kernel jumps. Stride 1 visits every position. Stride 2 visits every other one, halving the output in each dimension and quartering the number of positions.

MEASURED IN THE LAB

From `convolution.py`, one image, one kernel, four settings:

Setting	Output size	What it is for
stride 1, padding 1	32×32	the default: keep the resolution
stride 1, padding 0	30×30	shrinks slightly; edges under-used
stride 2, padding 0	15×15	deliberate downsampling
stride 4, padding 0	8×8	aggressive downsampling, as in AlexNet's first layer

COMMON MISTAKE

“Stride is just an optimisation to make things faster.”

It is a modelling decision that throws information away on purpose. Stride 2 means the layer never looks at half the positions, and the exact location of a feature within each 2×2 region is lost forever.

That loss is often what you want, because you usually care whether an eye is present rather than which precise pixel it starts at. But if you are doing something where precise position matters, such as segmenting a medical image pixel by pixel, aggressive striding destroys exactly the information you need, and architectures for those tasks (U-Net and its descendants) are built specifically to recover it.

• OPEN THE CONVOLUTION / FEATURE-MAP VISUALISER

Open tab 1, “Watch the kernel walk”, and press **Step** rather than **Walk** for the first dozen positions.

The middle panel shows the patch and the kernel side by side, with the actual multiplication written out underneath and the running total. The amber square on the left is where the kernel currently sits; the amber cell on the right is where the answer lands. Do about twenty single steps and the operation stops being abstract.

Then set the kernel to **vertical edge** and press **Finish**, and note **which** parts of the image lit up. Now switch to **horizontal edge** and finish again. Same image, same nine-number machinery, completely different map. Finally set padding to 0 and stride to 2 and watch the output size readout change to confirm the formula from Chapter 5.

WHY THIS MATTERS

Everything in Chapters 7 to 11 is a consequence of the arithmetic in this chapter, so an hour spent here saves several later.

In particular, the fact that the kernel does not change as it slides is the source of every property a CNN has: the small parameter count, the translation behaviour, the data efficiency, and the failure modes. Students who treat convolution as a black box find those properties arbitrary and have to memorise them. Students who have done the worked example above can derive them.

QUIZ WATCH · WEEK 6

Very likely: compute a convolution by hand given a small patch and kernel. Explain what a kernel whose weights sum to zero responds to. State the size formula and apply it. Explain the difference between padding and stride and give the purpose of each.

7 What a Convolution Layer Assumes

COVERS	The three assumptions, stated plainly, with the evidence for each
TIME	About 20 minutes; the heart of the CNN half of the week
IN BRIEF	Locality, stationarity and compositionality. Three sentences about the world, and a CNN is what you get when you write them as arithmetic.

A convolution layer encodes three beliefs. They are usually presented as a list to memorise. They are better understood as three claims that happen to be true about photographs and false about many other things.

7.1 Assumption one: locality

◆ THE ASSUMPTION: LOCALITY

The plain sentence: what a thing means is determined by what is near it, not by what is far from it.

In the operation: each output depends on a $k \times k$ patch and nothing else. A 3×3 kernel physically cannot see a pixel four steps away.

Why it is true for images: an edge, a corner, a texture are all defined by a handful of adjacent pixels. Whether this pixel is part of a boundary is decided by its immediate neighbours. The pixel in the opposite corner is essentially uninformative about it.

What breaks when it is false: if the real relationship in your data is between distant elements, a single convolution layer cannot express it, and the model must route the relationship through many layers of local summarising. Sometimes that works. Sometimes the signal is destroyed on the way.

7.2 Assumption two: stationarity, also called translation equivariance

◆ THE ASSUMPTION: STATIONARITY

The plain sentence: a pattern worth detecting in one place is worth detecting in every place, and it looks the same wherever it appears.

In the operation: the same kernel weights are used at every position. This is weight sharing from Chapter 4.

Why it is true for images: a cat's ear is ear-shaped whether it is top left or bottom right. The physics of edges does not change with position in the frame.

What breaks when it is false: if position genuinely matters, sharing forces one compromise detector on every location. Faces are a real example: in aligned face photographs the eyes are always in roughly the same place, and some specialised architectures deliberately **unshare** weights to exploit that.

This second assumption has a precise mathematical form, and it is one of the few places in this module where a property can be **proved** rather than measured.

KEY IDEA

Convolution **commutes with translation**. In symbols, for a shift operator S :

$$\text{conv}(S(x)) = S(\text{conv}(x))$$

Shift the input and the output shifts by the same amount, with nothing else changed. This is called **equivariance**: the output varies with the input in a predictable, structured way.

This is a property of the operation, not something the network learns from examples. A dense layer has no such property: to a dense layer, a shifted image is simply a different input vector, and every weight has to learn about it separately.

MEASURED IN THE LAB

From `equivariance_test()` in `convolution.py`. Both orders of operations are computed and compared:

Operation	Quantity measured	Result
convolution	$\max \text{conv}(S(x)) - S(\text{conv}(x)) $	0.00e+00
dense layer	$\max WS(x) - Wx $	1.987, or 60% of the output's scale

Exactly zero. Not small, not approximately zero: identical to the last bit, because the two computations are the same arithmetic in a different order.

The dense layer's output changes by 60% of its own scale. To it, the shifted image is an unrelated input.

IN PLAIN ENGLISH

Equivariance is **the promise that moving the thing moves the answer, and changes nothing else**.

If you have a machine that finds where the ripest apple is in a photograph, equivariance means: shift the photograph two centimetres to the right, and the machine reports a position two centimetres to the right. It does not suddenly decide a different apple is riper.

A dense network makes no such promise. It might report the same apple, if it saw enough shifted examples during training to learn that shifts do not matter. But it **learned** that from data, at a cost, and only for the shifts it happened to see. The convolution never had to learn it, because the arithmetic guarantees it.

MEASURED IN THE LAB

What that guarantee is worth, from `experiment_unseen_positions()`. All three models are trained with the target pattern appearing **only in the left third** of the image, then tested with it appearing **only in the right third**, where they have never seen anything at all:

Model	Parameters	Positions seen in training	Positions never seen
CNN, global pooling	700	1.000	0.998
CNN, flatten head	2,716	0.993	0.230
MLP	16,708	0.470	0.253

The first row is equivariance paying out. The detector learned on the left already works on the right, because it is the same nine numbers. Generalising to a region it has literally never seen costs it 0.002.

The second row is Chapter 9's warning, arriving early. Rows 1 and 2 have **identical convolutional layers**. They differ only in the last two lines of the architecture, and one of them is at chance.

7.3 Assumption three: compositionality

◆ THE ASSUMPTION: COMPOSITIONALITY

The plain sentence: complicated things are made of simpler things, and the simpler things are made of simpler things still.

In the operation: layers are stacked. Layer 1 sees raw pixels and finds edges. Layer 2 sees a map of edges and finds combinations of edges, which are corners and curves. Layer 3 finds combinations of those, which are textures and parts. Layer 8 finds objects.

Why it is true for images: the visual world genuinely is hierarchical. A face is made of eyes and a nose; an eye is made of curves; a curve is made of edges; an edge is made of a brightness change between adjacent pixels.

What breaks when it is false: if your data's structure is flat, depth buys you nothing but training difficulty. This is a large part of why deep networks rarely beat gradient-boosted trees on tabular data: there is often no hierarchy of features to discover.

MEASURED IN THE LAB

Compositionality has an unusually satisfying piece of evidence, and `learn_first_layer()` in `convolution.py` produces it.

Take image patches. Give them **no labels at all**. Ask only that each 3×3 patch be compressed to eight numbers and reconstructed. Nobody mentions edges.

What comes out are edge detectors, oriented at various angles, closely resembling the Sobel and diagonal kernels that humans designed by hand in the 1960s and 70s. `fig_learned_filters()` puts the learned filters directly above the hand-designed ones for comparison.

The reason this is more than a curiosity: the first-layer filters of AlexNet, of ResNet, of a student project, and of the mammalian visual cortex all look broadly alike. That convergence is evidence that edges are not an arbitrary human choice of feature. They are what natural images are actually built from, so any system compressing natural images finds them.

○ DEEPER DETAIL: Equivariance in general, and why it is a research programme

Translation is one symmetry. It is the one convolution handles, and handling it well is why CNNs worked.

But the world has other symmetries. A molecule's energy does not change if you rotate it. A galaxy's classification does not change if you reflect the image. A crystal's properties are unchanged under its particular symmetry group. In every case, the same argument applies: if a transformation should not change the answer, a model that is **guaranteed** not to change its answer under that transformation needs less data than one that must learn the invariance from examples.

This is the field of **equivariant** or **geometric** deep learning, and it is one of the most productive current areas, particularly in the sciences. Rotation-equivariant networks are standard in molecular property prediction; the AlphaFold family uses architectures built around the symmetries of three-dimensional space.

The connection to Week 5 is direct. Week 5 asks what happens when you build known science into a model rather than hoping it will be learned. Equivariance is the cleanest case: instead of a soft assumption like locality, you encode an exact mathematical symmetry that the underlying physics genuinely obeys, and the model cannot violate it even if the data is noisy.

WHY THIS MATTERS

These three assumptions are the reason CNNs displaced hand-designed computer vision, and knowing them tells you when **not** to reach for a CNN, which is more valuable than knowing when to.

A concrete example you may well meet. Somebody proposes treating tabular data as an “image” by arranging the columns in a grid and running a CNN over it. This sounds inventive and is almost always a mistake, and now you can say exactly why: the arrangement is arbitrary, so locality is false (adjacent columns are not related) and stationarity is false (the same detector applied to the age column and the postcode column is meaningless). You are asserting two false things and paying for them.

QUIZ WATCH · WEEK 6

Near certain: name the three assumptions and explain each in plain language. Explain the difference between equivariance and invariance (Chapter 9 completes this). Give a data type where each assumption fails, with a reason. The “arrange tabular data as an image” scenario is a very natural applied question.

8 Feature Maps, Channels, and What Filters Learn

COVERS	What the intermediate values in a CNN actually are, and what the layers end up detecting
TIME	About 15 minutes
IN BRIEF	A convolution layer is a bank of filters, not one filter. Each produces a channel, and the stack of channels is the layer's description of the image.

Chapter 6 described one kernel. A real layer has many, and the bookkeeping of channels confuses more students than the convolution itself.

8.1 One filter, one channel

Apply one kernel to an image and you get one feature map: a grid of numbers saying how strongly that kernel's pattern appears at each position. Apply 32 different kernels and you get 32 feature maps, stacked. Those 32 maps are the layer's **output channels**.

So a feature map is a **map of where one feature is**, and a channel is one such map. A layer's output is a stack of them.

IN PLAIN ENGLISH

Think of a stack of tracing paper laid over a photograph.

On the first sheet you have marked, at every position, how much of a vertical edge there is. On the second, how much of a horizontal edge. On the third, a diagonal. And so on for 32 sheets.

You have not thrown the image away. You have re-described it: instead of “how bright is each point”, you now have “how much of each of 32 patterns is at each point”. The next layer reads the stack of sheets, not the original photograph, and looks for patterns **in the sheets**, which is how you get from edges to corners.

8.2 Channels going in, channels coming out

Here is the part worth getting exactly right, because it is where parameter counts go wrong.

An input has C_{in} channels: 1 for greyscale, 3 for colour, or 64 if it came from a previous layer with 64 filters. Each filter in the current layer is **not** $k \times k$. It is $C_{\text{in}} \times k \times k$: it has a $k \times k$ patch of weights **for every input channel**, and it sums across all of them to produce one number.

So one filter looks at a small spatial window across **all** channels at once, and outputs a single channel. If you want C_{out} output channels, you need C_{out} such filters.

$$\text{params} = \underbrace{C_{\text{out}}}_{\text{how many filters}} \times \underbrace{(C_{\text{in}} \times k \times k)}_{\text{size of each filter}} + \underbrace{C_{\text{out}}}_{\text{one bias each}}$$

WORKED EXAMPLE: Counting parameters through three layers

A small network on a colour image:

Layer	Specification	Output channels	Parameters
1	Conv2d(3, 32, 3, padding=1)	32	$32 \times (3 \times 3 \times 3) + 32 = 896$
2	Conv2d(32, 64, 3, padding=1)	64	$64 \times (32 \times 3 \times 3) + 64 = 18,496$

3	Conv2d(64, 64, 3, padding=1)	64	$64 \times (64 \times 3 \times 3) + 64 = 36,928$
---	------------------------------	----	--

Total: 56,320 parameters, and this network can be applied to an image of any size.

Two things to notice. First, the parameter count **grows with depth**, not because the kernels get bigger but because C_{in} does. Layer 3's filters are $64 \times 3 \times 3 = 576$ weights each. Second, the common beginner's error is to compute layer 2 as $64 \times 9 + 64 = 640$, forgetting the input channels, which understates it by a factor of 32.

COMMON MISTAKE

“A 3×3 filter has 9 weights.”

Only in the first layer of a greyscale network. A 3×3 filter in a layer with 64 input channels has $64 \times 9 = 576$ weights. Forgetting the channel dimension is the most common parameter-counting error in the module and it is worth marks in the quiz.

8.3 What the layers actually learn

The empirical finding, consistent across architectures and datasets, is a hierarchy:

Depth	What the filters respond to	Receptive field
layer 1	oriented edges, colour blobs, simple gradients	a few pixels
layers 2 to 3	corners, junctions, curves, simple textures	tens of pixels
middle layers	repeated textures, patterns, object parts (an eye, a wheel, a letter)	a large fraction of the image
late layers	whole objects and configurations of parts	the whole image

This is compositionality from Chapter 7, observed rather than assumed. It is also the origin of **transfer learning**, which deserves stating plainly because it is the single most useful practical technique in the chapter.

✓ IN PRACTICE: Transfer learning, and why it follows from this chapter

Early layers learn edges and textures. Edges and textures are the same in medical scans, satellite photographs and holiday snaps, because they are properties of images rather than of subject matter. Only the late layers are specific to the task.

So: take a network trained on millions of general photographs, keep the early layers, replace the last layer or two, and train only those on your few thousand examples. You inherit a feature extractor that took a large laboratory weeks of compute to produce.

In practice this is the difference between needing a million labelled examples and needing a thousand. If you take one operational technique from Week 4 into a real project, take this one. It is also the clearest illustration of the week's thesis: the transferred layers are **reusable assumptions**, learned once and then supplied for free.

8.4 Activation functions, and why polarity matters

Between convolutions there is a non-linearity, almost always ReLU. Week 3, Chapter 15 covered why any non-linearity is needed: without one, stacked linear layers collapse into a single linear layer and depth buys nothing.

There is a second reason specific to convolution, and it is neater. Recall from Chapter 6 that an edge and its opposite produce responses of the same size and opposite sign. ReLU keeps positives and zeroes negatives, so each filter reports only the polarity it was built for. “Dark to bright, left to right” becomes a feature distinct from

“bright to dark, left to right”, and the next layer can build on them separately. Without the non-linearity, those two would partly cancel when combined downstream.

› OPEN THE CONVOLUTION / FEATURE-MAP VISUALISER

Open tab 2, “A bank of filters”. You are looking at one image and six filters producing six channels.

Set **After the convolution** to nothing (raw response) and note that each map has both blue and red regions, which are the two polarities of the same feature. Now switch to ReLU (keep positives) and watch half of each map go flat. That is polarity being separated.

Then switch pooling on and note the maps get smaller and blockier. Chapter 9 is about what that trade buys.

QUIZ WATCH · WEEK 6

Likely: given a layer specification with input and output channel counts, compute the parameters. Explain what a channel is and what a feature map is. Describe the hierarchy of features across depth. Explain transfer learning and why the structure of a CNN makes it work.

9 Pooling, Stride, and the Receptive Field

COVERS	How local operations add up to global understanding, and the difference between equivariance and invariance
TIME	About 18 minutes; the equivariance-against-invariance distinction is the most-missed point in the week
IN BRIEF	Depth turns a 3×3 view into a whole-image view. Pooling converts “the output moves with the input” into “the output does not care”.

A 3×3 kernel sees nine pixels. Somehow a CNN recognises an object spanning the whole frame. This chapter is how those two facts fit together.

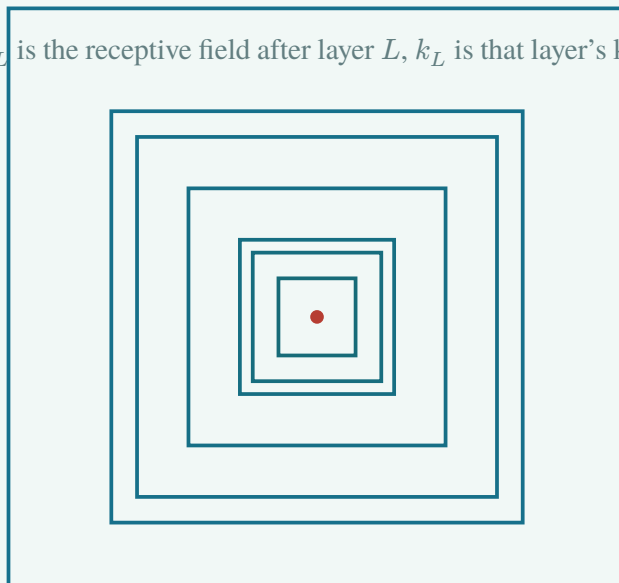
9.1 The receptive field

The **receptive field** of a unit is the region of the original input that can affect it. For a first-layer unit with a 3×3 kernel, it is 3×3 . Stack another 3×3 layer and each second-layer unit sees a 3×3 patch of first-layer units, each of which saw 3×3 pixels, so it sees 5×5 pixels. Add pooling or stride and the growth accelerates, because each step now moves further in the original image.

The recurrence, worth knowing:

$$r_1 = k_1, \quad r_L = r_{L-1} + (k_L - 1) \times \prod_{i < L} s_i$$

where r_L is the receptive field after layer L , k_L is that layer's kernel size and s_i are the strides of the earlier layers.



after	sees	pixels
L1	3×3	9
L2	5×5	25
L3	6×6	36
L4	10×10	100
L5	14×14	196
L6	16×16	256
L7	24×24	576
L8	32×32	1024

MEASURED IN THE LAB

From `receptive_field_table()`, for a typical small stack of 3×3 convolutions with 2×2 pooling after every second one:

After	Operation	Receptive field	Pixels seen
layer 1	conv 3×3	3×3	9
layer 2	conv 3×3	5×5	25
layer 3	pool 2×2	6×6	36
layer 4	conv 3×3	10×10	100

layer 5	conv 3×3	14 × 14	196
layer 6	pool 2×2	16 × 16	256
layer 7	conv 3×3	24 × 24	576
layer 8	conv 3×3	32 × 32	1,024

By layer 8 one unit depends on all 1,024 pixels of a 32 × 32 image. Nobody wrote a rule about the whole image. Global understanding was assembled out of purely local operations, which is compositionality doing its job.

IN PLAIN ENGLISH

The receptive field is **how much of the original picture one unit has any information about.**

A useful way to picture it: a chain of people passing summaries upward. The person at the bottom looks at nine pixels. The person above them hears from nine people at the bottom, so they have information about a wider area, though only in summarised form. Twenty levels up, somebody has information derived from the entire picture, without ever having looked at it directly.

This also explains a real failure mode. If your objects are larger than the receptive field of the layer making the decision, the network is being asked to identify something it cannot fully see. Checking the receptive field against the size of the thing you care about is a genuine debugging step, and it takes two minutes with the formula above.

9.2 Pooling, and what it is really for

Max pooling takes a small window, usually 2 × 2, and keeps only the largest value. **Average pooling** keeps the mean. Either way the output is half the size in each dimension.

Two things happen at once and they are worth separating:

1. **The output shrinks**, which saves computation and increases the receptive field faster.
2. **The exact position within the window is discarded**, which buys a small amount of tolerance to movement: a feature that shifts by one pixel inside a window produces the same output.

9.3 Equivariance against invariance

This is the distinction that most often goes unnoticed, and the lab result on it is stark.

KEY IDEA

Equivariance: move the input, and the output moves correspondingly. Convolution is equivariant to translation.

Invariance: move the input, and the output does not change at all. This is usually what you actually want from a classifier: a cat is a cat wherever it sits in the frame.

Convolution alone gives you equivariance, not invariance. Pooling converts **some** equivariance into invariance, locally. And crucially, **what you do at the end of the network decides whether the assumption survives to the output at all.**

Here is why that last sentence matters. Suppose you finish a CNN the traditional way: flatten the final feature maps into a long vector and feed it to a dense layer. That dense layer reads **positions**. Feature 7 at position (3, 4) arrives in a different input slot from feature 7 at position (9, 1), and the dense layer has separate weights for each. The convolutional trunk carefully maintained equivariance, and then the head threw it away.

The alternative is **global average pooling**: average each channel over every position, giving one number per channel. This asks “is this feature present anywhere?” and discards where. Now moving the input does not change the output at all, and the assumption is carried all the way to the prediction.

MEASURED IN THE LAB

The two heads, on identical convolutional layers, from `experiment_unseen_positions()`. Trained with the pattern only in the left third; tested with it only in the right third:

Model	Parameters	Positions seen	Positions never seen
CNN, global average pooling	700	1.000	0.998
CNN, flatten then dense	2,716	0.993	0.230

Both networks have the **same convolutions**. Both learned the pattern essentially perfectly where they saw it. One transfers to unseen positions almost losslessly; the other is at chance on a four-class problem.

The difference is two lines of architecture. An assumption is worth only what your architecture does with it **all the way to the output**, and a position-reading head discards it at the last moment.

COMMON MISTAKE

“CNNs are translation invariant.”

Said constantly, including in textbooks, and it is not right. Convolution is **equivariant**. Pooling adds a little local invariance. Global pooling gives you genuine invariance. A flatten-and-dense head gives you almost none, as the 0.230 above demonstrates.

If you say “invariant” when you mean “equivariant” in the quiz you will probably be forgiven. If you **believe** it, you will not understand why your model fails on shifted inputs, and you will look for the bug in the wrong place.

◉ DEEPER DETAIL: Global pooling, and the property it restores

Global average pooling has a second benefit that is easy to miss. Because it produces one number per channel regardless of the spatial size, the layer after it has a fixed input size **whatever the input image size was**.

This restores, at the level of the whole network, the size-independence that Chapter 3 showed for a single convolution layer. A CNN ending in global pooling can be applied to an image of any size. A CNN ending in flatten-and-dense is locked to exactly one input size, which is why older architectures required all images to be resized to precisely 224×224 .

This is why global pooling became standard after around 2015, and it is a good example of an architectural choice that looks like a minor detail and is actually an assumption with several consequences.

► OPEN THE CONVOLUTION / FEATURE-MAP VISUALISER

Open tab 3, “Depth and the receptive field”. Drag the **Layers** slider from 1 to 8 and watch the boxes grow outward from the red dot, with the table on the right giving the arithmetic for each step.

Then set **Pool every** to 1 and watch how much faster the field grows when you downsample aggressively, and to 4 to see it crawl. Compare the layer at which the field first covers the whole dashed square. That is the layer at which the network can first, even in principle, make a decision that depends on the entire image.

QUIZ WATCH · WEEK 6

Very likely: compute a receptive field for a given stack of layers. Distinguish equivariance from invariance with a definition of each and one example. Explain what pooling discards and what it buys. The “identical convolutions, different head, 0.998 against 0.230” result is an ideal applied question and you should be able to explain it in three sentences.

10 CNN Architectures, and What Each One Fixed

COVERS A short lineage, read as a sequence of problems and repairs rather than a list of names
TIME About 18 minutes
IN BRIEF Every famous architecture exists because the previous one broke in a specific way. Learn the problems and the names come free.

There is a temptation to memorise architectures as a list of names and dates. Resist it. The useful structure is a chain: each design solved a problem created by the last, and Week 3's closing lesson (every fix creates the next problem) applies exactly.

10.1 The lineage

Year	Name	The problem it faced	What it did about it
1989	LeNet	handwritten digits, tiny compute, almost no data	established the pattern: convolution, pool, repeat, then classify. Ran in production reading cheques
2012	AlexNet	could the pattern scale to real photographs?	depth, ReLU instead of sigmoid, dropout, two GPUs, and heavy augmentation. Week 3's Chapter 17 argument: 1986's algorithm on 2009's data and 2007's silicon
2014	VGG	kernel sizes were ad hoc and hard to reason about	use only 3×3 kernels, stacked. Two 3×3 layers see 5×5 with fewer parameters and an extra non-linearity. Simple, regular, and very large
2014	Inception	what kernel size is right when objects vary in scale?	do several sizes in parallel in the same layer and let the network weight them. Also 1×1 convolutions to reduce channels cheaply
2015	ResNet	deeper networks were performing worse , even on training data	skip connections. Discussed below, because the reasoning is the most instructive in the chapter
2017	MobileNet	none of this fits on a phone	separate spatial mixing from channel mixing (depthwise separable convolution), cutting cost by roughly the channel count
2020	Vision Transformer	do we need the convolutional assumptions at all?	replace them with attention. Works superbly given enough data, worse than a CNN without it. Chapter 21

10.2 ResNet, and the problem it actually solved

This is worth doing carefully, because the usual explanation is wrong.

By 2015 people could train networks 20 layers deep. When they tried 56 layers, performance got **worse**, and not only on test data: on **training** data too. That last detail rules out overfitting entirely. An overfitting model has low training error. This had high training error, which means it was an **optimisation** failure, not a generalisation one. It was also strange, because a 56-layer network can express everything a 20-layer network can: set the extra 36 layers to compute the identity function and you have the smaller network exactly. So the deeper network was strictly more capable and performed worse, which means gradient descent was failing to find a solution it demonstrably had available.

The fix was to make the identity easy to represent. A residual block computes

$$y = F(x) + x$$

so the layer learns a **correction** to its input rather than a whole new representation. If a layer has nothing useful to add, it can output zero, which is easy, rather than having to learn to reproduce its input exactly, which is not.

IN PLAIN ENGLISH

The problem was not that deep networks could not do the job. It was that they could not be **taught** the job.

An analogy. Ask somebody to copy a document by hand, then have the copy copied, thirty times over. Each copyist is competent, but each introduces small errors, and the thirtieth copy is unrecognisable. Nobody was incapable; the pipeline degraded the signal.

A skip connection is: pass the **original** along with the copy at every stage, and let each copyist write down only their corrections. Now if a copyist has nothing to add they add nothing, and the original survives to the end.

KEY IDEA

A skip connection gives the gradient a route back to earlier layers that does not pass through every intervening weight matrix. The derivative of $y = F(x) + x$ with respect to x is $F'(x) + 1$, and that $+1$ means the gradient can flow back even when F' is tiny.

This is Week 3, Chapter 15's vanishing gradient, addressed structurally. And it is the same idea as the LSTM's cell path in Chapter 15 of this booklet: **provide an additive route along which the gradient is not repeatedly multiplied by something small**. Two families, two decades apart, same insight.

WHY THIS MATTERS

ResNet's reasoning is a model of how to diagnose a problem properly, and the method transfers.

The critical step was noticing that **training** error had gone up. That single observation eliminated overfitting, capacity and data as explanations, and pointed at optimisation. Week 3, Chapter 9 gave you the train-test gap as a diagnostic; this is that diagnostic doing serious work on a real problem.

Skip connections are now in essentially everything, including every transformer, which is why this chapter matters for Chapter 19 as well.

◉ DEEPER DETAIL: Batch normalisation, and an honest note about why it works

Almost every architecture after 2015 includes normalisation layers, and they are worth understanding at least operationally.

Batch normalisation rescales each channel's activations to roughly zero mean and unit variance, using statistics computed across the batch, then applies learned scale and shift. The practical effects are well established: training tolerates higher learning rates, is much less sensitive to initialisation, and converges faster.

Why it works is genuinely still argued about. The original paper proposed that it reduces “internal covariate shift”, the drift in the distribution of each layer's inputs as earlier layers change. Later work (Santurkar and colleagues, 2018) presented evidence against that explanation and argued that the real effect is a smoother loss surface, which connects back to Week 3, Chapter 13.

This is worth knowing as an example of something important: a technique can be reliably useful while the field's account of **why** remains contested. It also matters practically that batch normalisation behaves differently in training and inference, and that it depends on batch size, which is why **layer** normalisation, computed per example rather than per batch, is used in transformers instead.

COMMON MISTAKE

“Deeper networks perform worse because they overfit.”

Not the case in the situation ResNet addressed. Training error rose, which is the signature of an optimisation failure. Overfitting means **low** training error and high test error.

Getting this backwards means reaching for regularisation, more data or a smaller model, none of which would have helped. The fix was architectural.

QUIZ WATCH · WEEK 6

Likely: explain the degradation problem ResNet solved and why the rise in **training** error rules out overfitting. State what a residual block computes and how it helps the gradient. Name one architecture and the specific problem it addressed. Batch normalisation's effects are fair game; its contested explanation is a good discussion prompt.

11 Where the CNN's Assumptions Fail

COVERS The honest limits, and how to tell in advance whether a convolution suits your data
TIME About 12 minutes
IN BRIEF A CNN's assumptions are true of photographs and false of many things people apply CNNs to. This chapter is the checklist.

Every architecture chapter should end with its failure modes, and most textbooks do not.

11.1 The four ways locality and stationarity go wrong

One: the input has no meaningful spatial arrangement. Tabular data. The columns are in whatever order the database happened to use. Adjacency is meaningless, so locality is false. This is the pixel-shuffle experiment, except that nobody had to shuffle anything: the data arrived shuffled.

Two: position genuinely matters. Stationarity says a feature means the same thing everywhere. Sometimes it does not. In an aligned medical scan, a shadow in the upper left lung is a different finding from the same shadow in the lower right. A globally-pooled CNN, which asks only “is this feature present anywhere”, will be unable to express that distinction. This is a real and consequential failure and the fix is architectural: keep spatial information in the head, or feed position in explicitly.

Three: the important relationships are long-range. Locality plus depth reaches the whole image eventually, but “eventually” costs layers, and information is summarised at every step. If the answer depends on comparing two distant regions in detail, a CNN has to squeeze that comparison through a bottleneck of local summaries. This is precisely the gap attention fills, and Chapter 17 begins there.

Four: the transformation you care about is not translation. Convolution is equivariant to shifts and to nothing else. Rotate an image 30 degrees, or scale it by half, and a CNN has no structural guarantee at all. It handles those cases only if it saw them during training, which is why rotation and scale augmentation is standard practice: you are teaching by example what the architecture does not know by construction.

MEASURED IN THE LAB

The clean statement of failure mode one, from `experiment_shuffle()`:

Pixel order	CNN	MLP
original	0.992	0.328
shuffled	0.255	0.333

And of the “you already solved it in preprocessing” case, from `experiment_no_payoff()`, on pre-centred digits:

Model	Test accuracy	Note
CNN	0.940	assumption buys nothing here
MLP	0.978	and costs something

▮ CASE FILE: Texture, not shape

A well-documented and initially surprising finding: CNNs trained on standard image datasets rely far more on **texture** than on **shape**. Geirhos and colleagues (2019) showed that an image of a cat with elephant-skin texture applied is classified as an elephant by standard CNNs, while humans overwhelmingly say cat.

Why this follows from Chapter 7. Texture is a **local** statistical property, exactly what stacked local operations detect most naturally. Shape is a **global** relational property, requiring integration across the whole object. The architecture's assumptions make texture the path of least resistance, so gradient descent takes it.

Two lessons, and the second is the one that generalises. First, a model can reach high accuracy by a route quite different from the one you assumed, and it will look fine on your test set because your test set has the same texture-shape correlation as your training set. Second, this is Week 2's argument arriving through a new door: the model learned a **shortcut** that happens to work on your data distribution and fails outside it. Week 2 called this a validity problem; here you can see its architectural cause.

✓ IN PRACTICE: Deciding in advance whether a convolution suits your data

Four questions, answerable before you write any code.

1. **Would shuffling the input positions destroy the meaning for a human?** If yes, adjacency matters and locality is plausible. If no, do not use a convolution.
2. **Does the same pattern mean the same thing at every position?** If yes, share weights. If position changes the meaning, either keep spatial information in the head or supply position explicitly.
3. **Is the signal local, or does it require comparing distant parts in detail?** Local favours convolution; long-range relations favour attention.
4. **What transformations should leave the answer unchanged?** Translation is free. Rotation and scale are not, and must come from augmentation or a specialised equivariant architecture.

If you can answer these four, you can defend your architecture choice in a sentence each, which is exactly what the Week 12 critical brief asks for.

WHY THIS MATTERS

Knowing an architecture's failure modes is what separates using a tool from cargo-culting it.

CNNs are applied to tabular data, to unordered sets and to arbitrary feature vectors surprisingly often, because they are famous and they are the default in tutorials. Every one of those applications asserts something false about the data and pays for it in accuracy, and the person who wrote it usually cannot say why the model is disappointing.

QUIZ WATCH · WEEK 6

Likely: name and explain two situations in which a CNN's assumptions fail. Explain the texture-over-shape finding and connect it to the locality assumption. Given a described dataset, argue for or against a convolutional architecture.

12 Sequences: Why Order Is a Different Kind of Structure

COVERS	What changes when the input arrives in an order, and why images and sentences need different assumptions
TIME	About 14 minutes; the pivot into the second half of the week
IN BRIEF	Sequences have a structure that is not spatial: length varies, order carries meaning, and dependencies can be arbitrarily far apart.

Everything so far assumed a fixed grid. A photograph is 224×224 and stays that way. Sequences break that in three ways at once, and each break rules out something.

12.1 The three things that change

One: the length varies. Sentences are three words or three hundred. A dense layer needs a fixed input size, so it cannot accept both. Any architecture for sequences must handle variable length, which immediately forces some kind of weight sharing.

Two: order carries meaning, and it is not the same as position in space. “The dog bit the man” and “the man bit the dog” contain identical words. In an image, swapping two distant pixels barely matters; in a sentence, swapping two words can reverse the meaning. Order is **the** structure, not a container for it.

Three: dependencies can be arbitrarily long-range and are not related to distance. This is the one that shapes the rest of the week.

Consider: “**The keys that I left on the table in the hall are missing.**” The verb “are” must agree with “keys”, eight words earlier. Nothing between them is relevant to that agreement, and the intervening words could be extended indefinitely: “the keys that I left on the table in the hall of the house we rented last summer are missing”. The dependency does not weaken with distance and cannot be bounded in advance.

IN PLAIN ENGLISH

In an image, things that matter to each other are usually near each other. In language, they are frequently not, and how far apart they are is decided by grammar rather than by any fixed window.

This is why the convolution’s core assumption fails here in a specific way. A width-3 kernel reading that sentence at the position of “are” can see “hall”, “in”, “the”. It cannot see “keys”. Not “sees it weakly”: cannot see it at all. Stack enough layers and the information can arrive, having been summarised at each step, but you have made a long-range grammatical fact into a game of telephone.

MEASURED IN THE LAB

The structural comparison, from `path_length_table()` in `sequences.py`. How many operations separate the first position from the last?

Sequence length	RNN	CNN with 3-wide kernels	Attention
10	10 steps	about 3 layers	1 step
100	100 steps	about 5 layers	1 step
1,000	1,000 steps	about 7 layers	1 step

A convolution reaches distant positions logarithmically, which sounds acceptable and is not the whole story: each of those layers is a lossy summary, and the information has to survive all of them.

Note also the column not shown: an RNN's steps must happen **one after another**, so they cannot be spread across a wide processor. Attention's can. Chapter 16 returns to this, because it turned out to matter more than anything about accuracy.

12.2 What structure a sequence architecture could exploit

Given the three changes, what assumption is available? The natural one is a rewriting of stationarity, moved from space to time.

◆ THE ASSUMPTION: STATIONARITY IN TIME

The plain sentence: the rule for updating your understanding as you read does not change as you read. What you do with word 40 is what you did with word 4.

In the operation: one weight matrix, applied at every step. Weight sharing across time, from Chapter 4.

Why it is plausible: grammar does not change halfway through a sentence. The way a reader incorporates a new word is broadly the same at the start and at the end.

What it buys: a fixed parameter count for any length, and the ability to read sequences longer than any seen in training.

What it costs: the same matrix at every step means the gradient becomes a **power** of one matrix, which is Chapter 14 and is much worse than it sounds.

There is a second assumption smuggled in alongside, and it is the one that eventually breaks recurrence.

◆ THE ASSUMPTION: THE PAST CAN BE SUMMARISED

The plain sentence: everything relevant about the first t items can be squeezed into one fixed-size summary.

In the operation: a hidden state vector h_t of fixed width, which is the **only** thing carried forward.

Why it is plausible: human reading feels like this. You do not hold every word in mind; you hold a running understanding.

What it costs: a fixed-size vector has a fixed capacity. If the task requires remembering more than it can hold, no amount of training helps. Chapter 16 measures exactly this, and the numbers are unambiguous.

WHY THIS MATTERS

Separating those two assumptions is worth doing, because they fail differently and get confused constantly.

Stationarity in time causes the **vanishing gradient**: a training problem, about signal reaching backwards. Gates (Chapter 15) address it.

The fixed-size summary causes the **bottleneck**: a capacity problem, about information reaching forwards. Gates do **not** address it, and attention does. Chapter 16 shows an LSTM failing for the second reason while its gradients are perfectly healthy, which is the clearest way to see that these are two different problems.

QUIZ WATCH · WEEK 6

Likely: name the three ways sequences differ from fixed grids and what each rules out. Give an example of a long-range dependency and explain why a small convolution cannot capture it. State the two assumptions a recurrent network makes and distinguish what each one costs.

13 The Recurrent Network

COVERS	Recurrence as weight sharing through time, written out
TIME	About 16 minutes
IN BRIEF	One weight matrix, applied once per step, carrying a running summary. Three lines of arithmetic, and every property follows from them.

13.1 The model

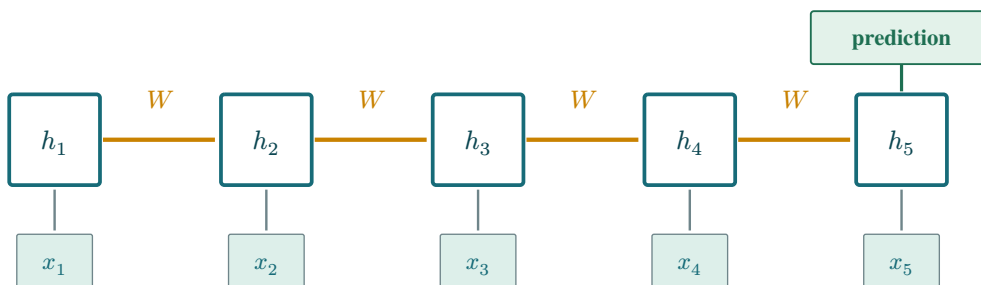
At each step t , take the new input x_t and the previous summary h_{t-1} , and produce a new summary:

$$h_t = \tanh(x_t W_x + h_{t-1} W_h + b)$$

Then, usually at the end, produce an output from the final summary:

$$\hat{y} = h_T W_o + b_o$$

That is the whole model. Note what is shared: there is exactly one W_x , one W_h and one b , used at every step.



Every W above is the same matrix. That is weight sharing across time.

IN PLAIN ENGLISH

A recurrent network is **a reader with a notepad of fixed size**.

It reads one word, updates the notepad, reads the next word, updates the notepad again. The rule for updating is always the same. At the end, it answers the question using only what is on the notepad.

Two things follow immediately, and they are the whole of the next four chapters. The notepad is a **fixed size**, so there is a limit to what can be on it. And the answer depends on the first word only through a chain of updates, each of which may have written over it.

13.2 The code, in nine lines

From `sequences.py`:

```
def forward(self, X):
    N, T, _ = X.shape
    H = np.zeros((N, T + 1, self.hidden))
    for t in range(T):
        H[:, t+1] = np.tanh(X[:, t] @ self.Wx + H[:, t] @ self.Wh + self.b)
    self.X, self.H = X, H
    return H[:, -1] @ self.Wo + self.bo
```

The loop is the recurrence. `self.Wh` inside the loop is the same array on every iteration, which is the weight sharing. And `H` stores every intermediate state, because the backward pass will need all of them, exactly as Week 3, Chapter 14 warned about memory.

MEASURED IN THE LAB

Parameter counts from the lab, for a 5-dimensional input, 24 hidden units and 4 output classes:

Model	Parameters	Depends on sequence length?
RNN	820	no
LSTM	2,980	no
dense network on the flattened sequence	grows as $5T \times 24$	yes, linearly

At $T = 160$ the dense network needs over 19,000 parameters in its first layer alone, and it can only ever accept sequences of exactly length 160. The recurrent models accept any length at all with the same 820 numbers.

WORKED EXAMPLE: Two steps of an RNN, by hand

Take a hidden size of 1 to make the arithmetic visible. Let $W_x = 1.0$, $W_h = 0.5$, $b = 0$, and start with $h_0 = 0$.

Feed the sequence $x = (1.0, 0.0, 0.0)$, which is “one important input, then nothing”.

$$h_1 = \tanh(1.0 \times 1.0 + 0.5 \times 0) = \tanh(1.0) = 0.762$$

$$h_2 = \tanh(1.0 \times 0 + 0.5 \times 0.762) = \tanh(0.381) = 0.364$$

$$h_3 = \tanh(1.0 \times 0 + 0.5 \times 0.364) = \tanh(0.182) = 0.180$$

The signal from step 1 is decaying: 0.762, then 0.364, then 0.180, roughly halving each time. After ten steps of nothing it would be around 0.0007.

Now note where that decay came from. It is $W_h = 0.5$, applied once per step. The memory’s half-life is fixed by a single weight, chosen once during training, and it is the same for every input and every dimension. Chapter 15’s entire contribution is to make that number something the network chooses per step.

• OPEN THE SEQUENCE MEMORY EXPLORER

Open [interactive/sequence-memory-explorer.html](#), tab 3, and drag the sequence length slider while watching the parameter table. The RNN and LSTM rows do not move; the dense row grows and its note points out the fixed-length restriction.

Read the amber caption under the diagram: every W is the same matrix. If you take one thing from this chapter, take that, because Chapter 14 is entirely a consequence of it.

QUIZ WATCH • WEEK 6

Likely: write down the RNN update equation and explain each term in words. Explain why the parameter count is independent of sequence length. Work through two or three steps by hand given small weights, and say what the decay rate depends on.

14 Backpropagation Through Time, and the Vanishing Gradient Again

COVERS	Why depth in time is worse than depth in layers, with the arithmetic and the measurement
TIME	About 20 minutes; the most important chapter in the recurrent half
IN BRIEF	The gradient from the end of a sequence to its start is a product of T copies of one matrix. Slightly small vanishes, slightly large explodes, and the usable band is a knife edge.

Week 3, Chapter 15 introduced the vanishing gradient in a deep feedforward network. This chapter is the same problem in a considerably worse form, and understanding **why** it is worse is the point.

14.1 Backpropagation through time

To train an RNN, unroll it and apply the chain rule, exactly as in Week 3. The loss at the end depends on h_T , which depends on h_{T-1} , which depends on h_{T-2} , and so on back to h_1 . So:

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_T} \times \prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}}$$

Each factor in that product is, from the update equation, a diagonal matrix of tanh derivatives multiplied by W_h . So the whole product behaves like W_h raised to the power T .

KEY IDEA

The gradient reaching step 1 from step T is a product of T factors, and **every one of those factors contains the same matrix** W_h .

This is the crucial difference from a deep feedforward network. There, the layers have **different** weight matrices, so the factors are unrelated: some may be above 1, some below, and the product can behave reasonably by luck.

Here it is one matrix, over and over. If its relevant scale is slightly below 1, the product decays exponentially. If slightly above, it grows exponentially. There is no averaging out, because there is nothing to average.

Depth in time is systematically worse than depth in layers, and this is why.

IN PLAIN ENGLISH

Multiply 0.9 by itself sixty times and you get about 0.0018. Multiply 1.1 by itself sixty times and you get about 304.

Those two numbers differ by 0.2 in the thing being multiplied, and by a factor of 170,000 in the result. That is the entire problem, and it is arithmetic rather than a bug.

If the number is a bit less than 1, the loss at the end of a sentence ends up depending on the first word by an amount too small to act on, so the model cannot learn the dependency. If a bit more than 1, the gradient is enormous and one update destroys the model, usually producing nan.

MEASURED IN THE LAB

From `gradient_through_time()` in `sequences.py`. This unrolls an actual random recurrent network, accumulates the true Jacobian, and reports its largest singular value:

Weight scale	After 5 steps	After 20 steps	After 60 steps	Verdict
0.5	6.748e-02	5.253e-06	6.956e-17	vanishes
0.9	1.192e+00	4.593e-01	1.492e-01	usable
1.0	1.336e+00	6.417e-01	1.741e-01	usable
1.2	2.379e+00	2.311e+00	4.737e+00	growing

Read the first row across. After 60 steps the gradient is 7×10^{-17} . In 32-bit arithmetic that is indistinguishable from zero for any practical purpose: the loss at step 60 is **numerically independent** of the input at step 1.

Read the column of verdicts down. The band between vanishing and exploding is narrow and it is not something you can arrange reliably in advance, because the effective scale changes as the weights train.

MEASURED IN THE LAB

And the same phenomenon during actual training, from `experiment_memory()`. The task is: see a symbol at step 1, ignore T steps of noise, report the symbol at the end. Chance is 0.250.

Sequence length	RNN test accuracy	RNN gradient at step 1
10	1.000	1.90e-06
40	1.000	2.65e-05
90	0.463	7.81e-18
160	0.227	1.59e-127

That last figure is not a typo. 10^{-127} . The network is being asked to learn a dependency for which the gradient signal has underflowed into oblivion, and unsurprisingly it performs at chance.

14.2 The two standard patches, and their limits

Gradient clipping addresses explosion. If the gradient's length exceeds a threshold, rescale it to that length, keeping its direction. Two lines of code, and it reliably prevents the nan. It does nothing whatsoever for vanishing, because there is no way to rescale zero into information.

A better optimiser helps more than you might expect, and this is worth being honest about because it complicates the standard story.

MEASURED IN THE LAB

An honest finding from building this lab. With plain gradient descent, the RNN failed the memory task at $T = 20$. With Adam (Chapter 22, and `optim.py`), the **same architecture** solved it perfectly up to $T = 40$.

Adam rescales each parameter's step by that parameter's own typical gradient size, so a gradient that is small but consistent still produces a full-sized step. It partially rescues a vanishing gradient by **renormalising** it.

It does not remove the problem. At $T = 90$ the RNN with Adam still fails (0.463), because by then the gradient is 10^{-18} and there is no consistent signal left to renormalise.

The lesson: the vanishing gradient is a real architectural problem, **and** a good optimiser moves the point at which it bites. Both halves are true, and a lot of writing on this topic states only the first.

► OPEN THE SEQUENCE MEMORY EXPLORER

Open tab 1, “The gradient through time”. The curve is computed live in the browser by the same method the Python lab uses: unroll a real random matrix, accumulate the Jacobian, take its largest singular value by power iteration.

Set the scale to 0.90 and the length to 20. The gradient survives, and the verdict reads “usable”. Now drag the length to 150 **without changing anything else**. Nothing about the model changed. The task became unlearnable, purely because the same matrix has been multiplied by itself 150 times.

Then push the scale to 1.1 and watch the opposite failure. Finding the band where neither happens, and noticing how narrow it is, is the exercise.

COMMON MISTAKE

“The vanishing gradient means the model learns slowly.”

It means the model does not learn **that particular dependency at all**, while quite possibly learning others perfectly well. This is what makes it hard to spot: the loss goes down, the model appears to be training, and it is simply blind to anything more than a few dozen steps back.

Week 3, Chapter 15 made the same point about deep sigmoid networks training their last layers and leaving the first ones near their initial values. Same failure, same disguise.

QUIZ WATCH · WEEK 6

Very likely: explain why an RNN’s gradient is a power of one matrix and why that is worse than a deep feedforward network’s product of different matrices. Given a per-step factor and a sequence length, compute the approximate gradient scale. Explain what clipping fixes and what it does not. State one thing a better optimiser does and does not do about vanishing.

15 LSTMs and GRUs: Gates as Learned Memory Control

COVERS	The fix, and why it is a fix rather than a workaround
TIME	About 18 minutes
IN BRIEF	A gate is a learned number between 0 and 1 that controls how much of something passes. The LSTM's key move is to make forgetting a decision rather than a side effect.

15.1 The idea in one sentence

In a plain RNN, the rate at which the past fades is fixed by the weights: one number, learned once, the same for every input and every dimension. In an LSTM, that rate is **computed fresh at every step from the current input**. That is the entire contribution. Everything else is the machinery required to make it work.

15.2 The equations, with the important line marked

$f_t = \sigma(\dots)$	forget gate: how much of the old memory to keep
$i_t = \sigma(\dots)$	input gate: how much of the new candidate to add
$o_t = \sigma(\dots)$	output gate: how much of the memory to reveal
$g_t = \tanh(\dots)$	the candidate value itself
$c_t = f_t \cdot c_{t-1} + i_t \cdot g_t$	the cell, updated ADDITIVELY
$h_t = o_t \cdot \tanh(c_t)$	the visible state

Each gate is a sigmoid, so each is a number between 0 and 1 for every dimension: 0 means “block completely”, 1 means “pass completely”. They are computed from x_t and h_{t-1} by ordinary weighted sums, which means they are **learned**.

The line that matters is the fifth. Look at what happens to the old cell c_{t-1} : it is **multiplied by a gate and added to**. It is not passed through a weight matrix and squashed.

KEY IDEA

In a plain RNN, information from the past passes through W_h and $\tanh$ at every step, so the gradient is multiplied by that matrix repeatedly, which is Chapter 14's problem.

In an LSTM, the cell path is $c_t = f_t \cdot c_{t-1} + \dots$, so the gradient along it is multiplied only by f_t at each step. And f_t is something the network **chooses**. If it learns $f \approx 1$, the gradient passes back almost unchanged for hundreds of steps.

This is why it is called a **constant error carousel** in the original paper, and why the term “gradient highway” stuck. It is the same structural trick as ResNet's skip connection from Chapter 10: provide an additive route along which the gradient is not repeatedly multiplied by something small.

IN PLAIN ENGLISH

A gate is a **tap**. A number between 0 (closed) and 1 (fully open) that decides how much gets through. The network learns when to open and close each tap, from the input it is currently looking at.

The three taps do three jobs. The **forget** tap decides how much of what I already know to keep. The **input** tap decides how much of what I have just seen to write down. The **output** tap decides how much of what I

know to say out loud right now, which is useful because you often want to hold something in mind without acting on it yet.

Compare a plain RNN, which has one fixed fading rate and no taps at all. It cannot decide that **this** input is worth remembering for a long time and **that** one is noise. It fades everything at the same rate, always.

WORKED EXAMPLE: Memory half-life, and why the forget bias matters

If the forget gate holds steady at f , then after n steps the fraction of the original signal remaining is f^n . The **half-life** is where $f^n = 0.5$:

$$n_{\frac{1}{2}} = \frac{\log(0.5)}{\log(f)}$$

Forget gate f	Half-life	1% remaining after
0.5	1.0 step	7 steps
0.8	3.1 steps	21 steps
0.9	6.6 steps	44 steps
0.95	13.5 steps	90 steps
0.99	69 steps	458 steps
0.999	693 steps	4,603 steps

This table explains a real result from building this lab, and it is a good illustration of how an initialisation choice can masquerade as an architectural fact.

It is standard to initialise the forget-gate bias to a positive number, so the LSTM starts out remembering. The commonly cited value is between 1 and 2. At bias 1.5, $\sigma(1.5) = 0.82$, giving a half-life of about 3.5 steps.

With bias 1.5, this lab's LSTM scored **worse than a plain RNN** on the 90-step memory task. That looked like an architectural result. It was a bad initialisation: the LSTM began unable to see 90 steps back and had to learn its way out of that starting point.

At bias 3.0, $\sigma(3.0) = 0.953$, a half-life of about 14 steps, and the same LSTM scores 1.000 on the same task.

MEASURED IN THE LAB

The corrected comparison, from `experiment_memory()`. Both models use Adam, the same data, and the same number of epochs. Chance is 0.250.

Length	RNN accuracy	RNN gradient at step 1	LSTM accuracy	LSTM gradient at step 1
10	1.000	1.90e-06	1.000	1.92e-08
40	1.000	2.65e-05	0.997	5.21e-09
90	0.463	7.81e-18	1.000	8.93e-06
160	0.227	1.59e-127	0.250	7.61e-08

Row three is the argument. The RNN's gradient has collapsed to 10^{-18} and it is near chance; the LSTM's is 10^{-6} and it is perfect.

Row four is the more useful lesson, and it is why this table is printed in full. At 160 steps the LSTM's gradient **still survives**, 10^{-8} against the RNN's 10^{-127} , and yet the LSTM also fails. It was also tried with 220 epochs and a higher forget bias, and still failed.

So: **a surviving gradient is necessary but not sufficient**. Gates fix signal propagation. They do not, by themselves, make a long dependency easy to find. Anybody who tells you LSTMs solved long-range dependencies is overstating it, and this row is the evidence.

15.3 GRUs, and the honest comparison

The **gated recurrent unit** does the same job with two gates instead of three, merging the forget and input gates into one (“how much to replace”) and dropping the separate cell. It has roughly three quarters of the parameters.

Which is better? Empirically, it depends on the task and the difference is usually small. The GRU is cheaper and often trains faster; the LSTM sometimes wins on tasks needing very long memory. This is a genuine case where the honest answer is “try both”, and it is worth knowing that such cases exist, because most of the literature is written as though every choice has a principled answer.

◉ DEEPER DETAIL: Why sigmoid, specifically, for a gate

A gate needs to be a number between 0 and 1, because it is a fraction of something to let through. The sigmoid maps any real number into exactly that range, so it is the natural choice.

But there is a cost, and it is Week 3, Chapter 15’s cost. The sigmoid’s derivative peaks at 0.25 and approaches zero when saturated. A gate that has learned to be firmly open ($f \approx 1$) has almost no gradient **with respect to its own weights**, so it becomes slow to change its mind.

This is a real effect and it is why forget-gate initialisation matters so much: you are choosing where on the sigmoid the gate starts, and a gate that starts saturated in the wrong direction may take a very long time to move. It is also why the worked example above matters more than it looks: an apparently arcane initialisation constant determines whether your model can learn the task at all.

▸ OPEN THE SEQUENCE MEMORY EXPLORER

Open tab 2, “What a gate does”. The input stream has one important value at step 3 and noise afterwards.

Set the forget gate to 0.5 and watch the cell value: the spike at step 3 is gone within a few steps. Now drag it to 0.95 and watch the spike persist to the end of the sequence. The half-life table on the right highlights the row you are currently on, and the panel underneath computes what fraction of step 3 survives to step 39.

Then set the input gate to 0 and note that nothing is ever written down, and the output gate to 0 and note that the cell fills up while the visible state stays flat, which is exactly the “hold something in mind without acting on it” behaviour.

WHY THIS MATTERS

Gates are the idea, not the specific equations. What you should carry forward is: **when a signal has to survive many steps, give it a route that is added rather than repeatedly transformed, and let the model control the rate**.

That sentence describes the LSTM cell, the ResNet skip connection, the residual stream in every transformer, and the gating in modern state-space models such as Mamba. Four architectures across thirty years, one insight. Memorising six equations is much less useful than being able to recognise that pattern when you next meet it.

QUIZ WATCH · WEEK 6

Very likely: name the three gates and say what each controls. Write the cell update and explain why the additive form helps the gradient. Compute a memory half-life from a forget-gate value. Explain the difference between what gates fix and what they do not, using the 160-step result.

16 Where Recurrence Fails

COVERS	Two limits, one of which gates cannot fix, and the hardware argument that decided the matter
TIME	About 15 minutes; the bridge to attention
IN BRIEF	The bottleneck is a capacity problem, not a gradient problem. And recurrence cannot be parallelised, which turned out to matter more than accuracy.

16.1 Limit one: the fixed-size bottleneck

Chapter 12 flagged this as the second assumption: everything relevant about the past must fit in one fixed-size vector. Gates do not address it at all. A gate controls **what** goes into the state; it does not make the state bigger.

The failure is easiest to see in translation, which is where it was first felt acutely. An encoder reads a sentence and compresses it into one vector; a decoder generates the translation from that vector alone. For a short sentence, fine. For a paragraph, you are asking a fixed number of floating-point values to hold everything relevant about arbitrarily much text, and the performance of early systems fell off sharply with sentence length.

MEASURED IN THE LAB

The bottleneck isolated, from `experiment_lookup()` in `attention.py`. The task: a sequence of key-and-value pairs, then a query key; report the matching value. Every pair is needed, because the query could be about any of them. Chance is 0.250.

Sequence length	Pairs to remember	LSTM (2,980 to 9,092 params)	Attention (4,388 params)
4	3	0.550	0.995
9	8	0.362	1.000
17	16	0.340	0.998
25	24	0.278	0.995

Read the LSTM column down: it degrades steadily to chance as the number of pairs grows. Read the attention column down: it does not move at all.

This is **not** undertraining. The LSTM was given a more generous budget than the attention model, and was also tried at a higher learning rate for 150 epochs, scoring 0.270 instead of 0.278 at length 25. It is a structural mismatch: the number of things to remember grows and the state does not.

Note especially that this failure has **nothing to do with gradients**. The gradients here are healthy. The information simply does not fit. This is why Chapter 12 insisted on separating the two assumptions.

IN PLAIN ENGLISH

The bottleneck is **being allowed one index card**.

If I read you twenty name-and-phone-number pairs and then ask for one specific person's number, you cannot do it from one index card, no matter how good your handwriting or how carefully you decide what to write. There is not enough room, and no amount of practice creates room.

Attention's answer is not a better index card. It is to keep all twenty slips of paper and look up the one you need when asked. That is a completely different arrangement, and it is why the next chapter is a change of kind rather than a refinement.

16.2 Limit two: strictly sequential computation

This is the limit that actually decided the field, and it is not about accuracy at all.

To compute h_t you need h_{t-1} . So the steps must happen in order, one after another. A sequence of length 1,000 requires 1,000 sequential operations that cannot be overlapped.

Modern accelerators are **wide**, not fast: a GPU has thousands of cores and gets its speed from doing many things simultaneously. An architecture that forbids simultaneity cannot use the hardware. A transformer computes all positions at once, so a length-1,000 sequence is one large matrix multiplication, which is precisely what the silicon is built for.

KEY IDEA

Attention did not replace recurrence because it was more accurate on a fixed budget. It replaced recurrence because it **could be parallelised**, which meant it could be trained on far more data in the same wall-clock time, and more data on a weaker assumption turned out to beat less data on a stronger one.

This is Week 3's argument about mini-batches, at a much larger scale: **the operation that wins is the operation that suits the hardware**. Chapter 21 develops the consequence, and the paper that made it explicit was titled, pointedly, "Attention Is All You Need".

MEASURED IN THE LAB

From `path_length_table()`, with the column that matters:

Length	RNN path	Attention path	Parallelisable?
10	10 steps	1 step	RNN no, attention yes
100	100 steps	1 step	RNN no, attention yes
1,000	1,000 steps	1 step	RNN no, attention yes

A note on how quickly this happened

It is worth registering the speed of this transition, because it is unusual and it is recent enough to be instructive.

In 2016, recurrent networks with attention bolted on were the state of the art in machine translation, and LSTMs were the obvious choice for essentially any sequence problem. The transformer paper appeared in 2017 and removed the recurrence, keeping only the attention. Within about three years, recurrence had largely disappeared from new work in language.

Two things are worth taking from this. First, the winning idea was already present as a component of the previous generation: attention existed as an **addition** to recurrent models before it became the whole model. Progress often looks like deleting the part everyone assumed was essential. Second, the argument that won was substantially about hardware, which should make you sceptical of any claim that architectures are selected purely on their merits as models.

Also worth noting: recurrence is not dead. State-space models such as Mamba are recurrent in form and have been designed specifically to be parallelisable during training, which addresses limit two directly. Chapter 21 mentions where that stands.

WHY THIS MATTERS

These two limits are why the second half of the week exists, and knowing which is which prevents a common diagnostic error.

If your sequence model fails on long-range dependencies, there are two very different possible causes. Either the **gradient** is not reaching back, in which case gates, clipping and better optimisation are the right responses. Or the **information** does not fit in the state, in which case none of those will help and you need an architecture that keeps everything and looks things up.

The lookup experiment above shows the second case with healthy gradients throughout. Being able to tell them apart is a genuine engineering skill.

QUIZ WATCH · WEEK 6

Very likely: explain the fixed-size bottleneck and why gates do not fix it. Explain why an RNN cannot be parallelised and why that mattered more than accuracy. Distinguish the two failure modes and give the appropriate response to each.

17 Attention: Relations as a First-Class Object

COVERS	The change of kind, and why a weaker assumption turned out to win
TIME	About 16 minutes
IN BRIEF	Attention makes the relationship between two positions the thing being computed. Distance stops mattering, and content decides relevance.

Convolution assumes what matters is **nearby**. Recurrence assumes what matters arrives **in order** and can be summarised. Attention assumes something much weaker and considerably stranger.

◆ THE ASSUMPTION: CONTENT-BASED RELEVANCE

The plain sentence: any position might be relevant to any other, and which ones actually are is decided by what they contain, not by where they sit.

In the operation: every position computes a match score against every other position, and the scores come from the values themselves.

Why it is plausible for language: the pairs that matter in a sentence are not the pairs that are close together. A verb needs its subject, a pronoun needs its referent, a closing bracket needs its opening one, and none of those distances are bounded.

What it costs: computation that grows with the **square** of the length, and no knowledge whatsoever of what order the tokens arrived in until you add it separately. Chapters 19 and 20.

That is a **weaker** assumption than either of the others, and weaker assumptions are usually worse: Chapter 2 established that assumptions are worth data, so giving one up should cost you. It did. Attention won anyway, for two identifiable reasons: the assumption it gave up was **false** for language, and it could be parallelised, which bought back the data it needed.

17.1 The soft dictionary

The cleanest way to understand attention is as a lookup that has been made continuous.

In a Python dictionary you supply a key, and you get exactly one value back, or an error. It is exact, discrete and not differentiable, so you cannot put it in a network trained by gradient descent.

Attention is the smoothed version. You supply a **query**. Every position offers a **key**. Each key reports how well it matches the query, those scores are turned into fractions that sum to 1, and you get back a **blend** of all the values, weighted by match quality. Nothing is retrieved exactly and nothing is entirely absent, which is precisely what makes it differentiable.

IN PLAIN ENGLISH

Attention is **asking a room full of people a question and averaging their answers, weighted by how relevant each person says they are.**

You call out: “I am a plural verb and I need my subject.” Everybody in the room reports how well they fit that description. The plural nouns say “strongly”; the prepositions say “not at all”. Then you take a weighted average of what everybody knows, with the plural nouns dominating.

Two features fall out of this picture. It does not matter how far away the relevant person is, because they answer directly rather than passing a message along. And nobody has to decide in advance who will be relevant, because the matching happens fresh for each question.

MEASURED IN THE LAB

A worked example small enough to check by hand, from `worked_example()` in `attention.py`. Six tokens, four dimensions, embeddings chosen so that “keys” and “are” share a “plural” direction. After softmax, every row sums to 1:

	the	keys	that	I	left	are
the	0.17	0.17	0.17	0.17	0.17	0.17
keys	0.05	0.38	0.05	0.08	0.05	0.38
that	0.17	0.17	0.17	0.17	0.17	0.17
I	0.14	0.21	0.14	0.15	0.14	0.21
left	0.07	0.42	0.06	0.34	0.06	0.06
are	0.02	0.74	0.01	0.12	0.01	0.1

Rows are queries, columns are keys, and every row adds to exactly 1. The bottom row is the verb “are” finding its subject “keys”, four positions away, while giving almost nothing to the adjacent “left”.

MEASURED IN THE LAB

The bottom row is the payoff. The verb “are” gives **0.735** of its attention to “keys”, four positions away, and **0.013** to “left”, which is immediately adjacent.

Distance did not decide it; content did. And note what a width-3 convolution positioned at “are” could see: “on”, “the”, “table”. It cannot see “keys” at all. Not weakly: not at all.

WHY THIS MATTERS

This chapter is where the week’s thesis becomes subtle, and the subtlety is worth having.

Up to now the story has been simple: matching assumptions win. Attention appears to contradict that, because it assumes **less** and wins. The resolution is that both halves of Chapter 2 still hold.

Attention gave up an assumption that was **false for language** (that relevance follows proximity), which was a gain, not a loss. And it paid the predictable price for having a weaker prior overall: it needs a great deal of data. Chapter 21 shows that price being paid explicitly, with the Vision Transformer performing **worse** than a CNN on small datasets and better on enormous ones.

So the rule is not “strong assumptions are good”. It is: **the value of an assumption is exactly the extent to which it is true, and when you cannot make a true strong assumption, you must pay in data instead.**

QUIZ WATCH · WEEK 6

Likely: explain attention as a lookup and say what makes it “soft”. Explain why a weaker assumption can still win, in terms of truth of the assumption and cost in data. Give an example of a long-range dependency and explain why convolution cannot express it.

18 Self-Attention, Step by Step

COVERS	The three matrices, the scores, the softmax, and the weighted sum, with numbers
TIME	About 22 minutes; do the worked example
IN BRIEF	Queries, keys and values are three different projections of the same input. Everything else is a dot product and a softmax.

18.1 The operation

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

That formula intimidates people out of proportion to its content. Read it as four instructions.

- 1. Project.** From each token's vector, compute three different vectors by three learned matrices: a query $q = xW_Q$, a key $k = xW_K$, and a value $v = xW_V$.
- 2. Score.** For every pair of positions, take the dot product of one's query with the other's key. A dot product is large when two vectors point the same way, so it is a similarity measure.
- 3. Normalise.** Divide by $\sqrt{d}$, then apply softmax along each row, turning each row of scores into fractions summing to 1.
- 4. Mix.** Each position's output is the weighted average of all the **value** vectors, using those fractions.

18.2 Why three projections and not one

This is the question that most deserves an answer and most rarely gets one. Why not just use the token's vector for all three roles?

Because the three roles are genuinely different, and forcing them to be the same vector would confuse them.

- The **query** expresses what this token is **looking for**. For a verb: "a subject, plural".
- The **key** expresses what this token **offers** as a match target. For a noun: "I am a subject, plural".
- The **value** expresses what this token **contributes** if attended to, which may be something else entirely: its meaning, its tense, its referent.

IN PLAIN ENGLISH

Think of a noticeboard.

Your **query** is the advert you post: "wanted, one plumber". Your **key** is the label on your own card: "I am a plumber". Your **value** is what somebody actually gets if they hire you: your skills and your van.

These are three different things, and a system that used one vector for all three would be forced to say that what you are looking for, how you describe yourself, and what you provide are all identical. They are not. Separating them is what lets a verb search for a subject while **offering itself** as a target for something else entirely.

WORKED EXAMPLE: Self-attention on three tokens, entirely by hand

Take $d = 2$ so the arithmetic is visible. Three tokens with these embeddings:

$$x_1 = (1, 0), \quad x_2 = (0, 1), \quad x_3 = (1, 1)$$

Let $W_Q = W_K = W_V = I$ (the identity), so $q = k = v = x$ for each token. Now compute the attention for token 3.

Scores. $q_3 = (1, 1)$.

$$q_3 \cdot k_1 = (1)(1) + (1)(0) = 1$$

$$q_3 \cdot k_2 = (1)(0) + (1)(1) = 1$$

$$q_3 \cdot k_3 = (1)(1) + (1)(1) = 2$$

Scale by $\sqrt{d} = \sqrt{2} \approx 1.414$:

$$\frac{1}{1.414} = 0.707, \quad 0.707, \quad \frac{2}{1.414} = 1.414$$

Softmax. Exponentiate: $e^{0.707} = 2.028$, $e^{0.707} = 2.028$, $e^{1.414} = 4.113$. Sum is 8.169. So:

$$w = (0.248, 0.248, 0.503)$$

Check: they sum to 0.999, which is 1 up to rounding. Good.

Mix.

$$\text{out}_3 = 0.248(1, 0) + 0.248(0, 1) + 0.503(1, 1) = (0.751, 0.751)$$

Token 3 attended mostly to itself (0.503), because its query pointed most strongly at its own key, and split the rest evenly between the other two.

That is a complete self-attention computation. Everything in a transformer is this, at larger d , with learned projections instead of the identity, and repeated many times.

18.3 Why divide by the square root of the width

A detail that looks arbitrary and is not.

A dot product of two d -dimensional vectors with unit-scale entries has a magnitude that grows like $\sqrt{d}$. So in a model with $d = 512$, raw scores would be roughly 23 times larger than in a model with $d = 1$.

Large scores are a problem because softmax is exponential. If one score is much larger than the others, the softmax saturates: that weight goes to nearly 1 and the rest to nearly 0. And when softmax is saturated, its **gradient** is nearly zero, so nothing can be learned. This is Week 3, Chapter 15's saturation problem, arriving in a new place.

Dividing by $\sqrt{d}$ cancels the growth, keeping scores at a sensible scale regardless of model width. It is a scaling fix for a numerical problem, and knowing that stops it being one more thing to memorise.

• OPEN THE ATTENTION VISUALISER

Open [interactive/attention-visualiser.html](https://interactive.attention-visualiser.html), tab 2, “One query, step by step”, and click on a verb such as “are”.

The four stages are drawn as bar charts: raw dot products, then scaled, then after softmax, then the resulting output vector. The panel on the right writes the arithmetic out in full for the strongest link, including the check that the weights sum to exactly 1.

Then find the **Sharpness** slider and drag it to the far right. Watch one weight go to 1 and the rest to 0. That is saturation, and it is exactly what the $\sqrt{d}$ division exists to prevent. Now drag it to the far left and watch every row flatten to a uniform average, which is what an untrained model does before it has learned anything worth attending to.

◉ DEEPER DETAIL: The backward pass, and the one unfamiliar piece

The lab writes attention's backward pass out by hand in `AttentionClassifier.backward`, and it is worth knowing that only one step is new.

Matrix multiplications differentiate as in Week 3. The single unfamiliar piece is the softmax. Its Jacobian is a full matrix, and building it explicitly would be wasteful. In practice, if $p = \text{softmax}(s)$, then:

```
d_scores = W * (dW - (dW * W).sum(axis=-1, keepdims=True))
```

which computes the same thing without ever materialising the Jacobian. This is the same kind of collapse as Week 3's cross-entropy gradient reducing to $p - y$: a mathematically bulky object turning out to have a compact form.

The lab gradient-checks this against a numerical derivative and reports a maximum relative error of **4.8e-08**. Week 3, Chapter 14's rule holds: a wrong derivative fails silently, so check it once.

COMMON MISTAKE

“Attention weights show what the model is basing its decision on.”

Attractive, widely repeated, and not reliable. Attention weights show where information was **mixed from** at one layer, in one head. They are not an explanation of the model's reasoning, for several reasons: there are many heads and many layers, information mixed at layer 1 may be irrelevant by layer 12, and it is possible to change the weights substantially while barely changing the output.

Jain and Wallace's "Attention is not Explanation" (2019) made this case directly, and there was a rebuttal ("Attention is not not Explanation"), so this is genuinely contested rather than settled. The defensible position: attention maps are a useful **diagnostic** and **suggestive** of structure. Presenting them to a stakeholder as the reason for a decision is the Week 2 mistake of offering an explanation that has not been validated as one.

QUIZ WATCH · WEEK 6

Very likely: state the attention formula and explain each of the four stages. Explain why there are three separate projections and what each represents. Compute a small self-attention by hand. Explain why the scores are divided by $\sqrt{d}$. The "attention is not explanation" point is a strong discussion prompt.

19 Multi-Head Attention, Position, and the Transformer Block

COVERS	The three additions that turn one attention operation into a working architecture
TIME	About 20 minutes
IN BRIEF	One head can express only one notion of relevance. And attention has no idea what order the tokens came in, which is a mathematical fact with a required fix.

19.1 Multi-head attention

A single attention operation produces one set of weights per position, so it must express everything it cares about in a **single weighted average**. Language does not permit that. A verb simultaneously needs its subject, its object, its tense marker and its clause boundary, and blending those into one average loses all four.

The fix costs nothing. Take the model width d , cut it into h slices, and run attention independently in each slice with its own projections. Concatenate the outputs. Total parameters and computation are essentially unchanged, and you now have h independent notions of relevance.

IN PLAIN ENGLISH

One head is **one question asked of the room**. Multi-head attention is **asking eight different questions at once and keeping all eight answers**.

“Who is my subject?” “Who is my object?” “Which clause am I in?” “What tense are we in?” Each gets its own answer instead of all four being averaged into mush.

In trained transformers, heads reliably specialise, and some are interpretable enough to name: heads that attend to the previous token, heads that track syntactic dependencies, heads that match brackets, and the “induction heads” that copy patterns seen earlier in the context.

• OPEN THE ATTENTION VISUALISER

Open tab 4, “Many heads”. Four heads on one sentence, each with its own projections.

Compare head 1 (agreement) with head 4 (previous word). The mean-distance figure under each is the summary: one head is entirely about content and ignores distance, the other entirely about distance and ignores content. Both are **the same operation** with different projection matrices, which is the point. Attention has no built-in notion of what matters; it has a built-in notion of looking things up by similarity, and what counts as similar is learned.

19.2 Attention cannot see order

Now the property that surprises people, and it is not a subtlety but a hard mathematical fact.

KEY IDEA

Self-attention is **permutation equivariant**. Shuffle the input tokens and the outputs come back shuffled the same way, with nothing else changed.

Nothing in $\text{softmax}(QK^T)V$ refers to **where** anything is. Every position is treated identically; only content matters. So attention computes a function of the **set** of tokens.

It follows that a bare attention layer cannot distinguish “dog bites man” from “man bites dog”. This is not a limitation to be worked around cleverly. It is why every transformer ever shipped adds position information before the first attention layer.

MEASURED IN THE LAB

From `permutation_test()` in `attention.py`. Shuffle the tokens, run attention, un-shuffle the outputs, and compare against running attention on the original:

Input	Max difference	Verdict
embeddings only	8.88e-16	zero to machine precision: order is invisible
embeddings plus positional encoding	4.192	order now changes the answer

That first number is not “small”. 8.88×10^{-16} is the rounding error of 64-bit floating point. The two computations are the same arithmetic in a different order.

19.3 Positional encoding

The fix: add something to each token’s vector that identifies its position, before the first attention layer.

The original transformer used sinusoids:

$$\text{PE}(t, 2i) = \sin\left(\frac{t}{10000^{2i/d}}\right), \quad \text{PE}(t, 2i + 1) = \cos\left(\frac{t}{10000^{2i/d}}\right)$$

IN PLAIN ENGLISH

Why not just write the position number into a slot? Because a plain counter grows without bound, so position 5,000 would be a value of 5,000 sitting among numbers around 1, swamping every semantic dimension.

The sinusoidal scheme is **a set of clocks running at different speeds**. Each dimension oscillates at its own rate: some fast, some very slow. Every value stays between minus 1 and 1 at every position, and the **pattern across dimensions** is unique for each position, exactly as a set of clock hands at different speeds gives a unique reading at each moment.

It has a second, cleverer property: because a fixed shift in position corresponds to a fixed rotation of each sinusoid, the model has an easy route to **relative** position, which is usually what grammar cares about. “Three words before me” is a more useful fact than “position 47”.

Modern models mostly use **learned** embeddings or **rotary** position embeddings (RoPE), which encode relative position more directly and extrapolate better to longer sequences. They all exist for the reason in the table above.

► OPEN THE ATTENTION VISUALISER

Open tab 3, “Attention cannot see order”. Press **Shuffle the tokens** and compare the left and right panels: the third panel is the shuffled result mapped back to the original order, and it is identical to the first.

The readout says “order invisible” and gives the difference as 0.00e+0. Now tick **add positional encoding** and shuffle again. The number becomes large and the verdict changes. That is the entire justification for positional encoding, in one toggle.

19.4 The transformer block

A transformer is one block, repeated. Each block is:

1. **Multi-head self-attention**, which mixes information **between** positions.
2. A **feed-forward network**, two dense layers applied to each position **independently**, which processes information **within** each position.
3. A **residual connection** around each of those, which is Chapter 10’s skip connection.
4. **Layer normalisation**, computed per example rather than per batch.

KEY IDEA

The division of labour is the thing to remember: **attention moves information between positions; the feed-forward network transforms it within a position**. Alternating those two is the whole design.

The feed-forward part is where most of the parameters live, typically about two thirds. This is worth registering, because people describe transformers as “attention models” and then discover that most of the weights are in ordinary dense layers. Chapter 3’s closing point applies: dense layers did not go away, they stopped being asked to discover structure that could be built in instead.

DEEPER DETAIL: Causal masking, and the two ways to use a transformer

One detail decides whether a transformer can be used to generate text.

A **bidirectional** or encoder-style transformer lets every position attend to every other, including later ones. Good for understanding a complete input: classification, retrieval, and the BERT family.

A **causal** or decoder-style transformer masks out future positions, by setting those scores to negative infinity before the softmax so they receive zero weight. Now position t can see only positions up to t . This is what makes next-token prediction a valid training objective: without the mask, predicting the next word would be trivial, because the model could look at it.

Every generative language model you have used is causal, and the mask is roughly one line of code. It is a good example of an assumption expressed not through connectivity but through **which connections are switched off**, and it is available as a toggle in the Attention Visualiser’s controls.

QUIZ WATCH · WEEK 6

Very likely: explain why one attention head is insufficient and what multi-head attention does about it. State that attention is permutation equivariant and explain the consequence. Explain why positional encoding uses sinusoids rather than a counter. Name the components of a transformer block and say what each does. Explain causal masking and why generation requires it.

20 What the Transformer Assumes, and What It Gives Up

COVERS The honest accounting: the bill for a weak prior
TIME About 14 minutes
IN BRIEF Quadratic cost, no built-in notion of order or locality, and a large appetite for data. All three are consequences of the same design choice.

20.1 The bill

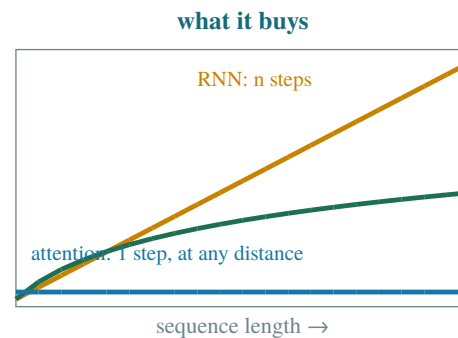
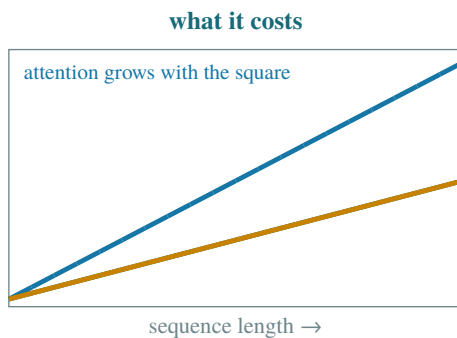
MEASURED IN THE LAB

From `cost_table()` in `attention.py`, with model width $d = 512$ and a 3-wide convolutional kernel:

Length	RNN ops	CNN ops	Attention ops	Attention / RNN
64	1.68e+07	5.03e+07	1.89e+07	1.1x
512	1.34e+08	4.03e+08	2.68e+08	2.0x
4,096	1.07e+09	3.22e+09	9.66e+09	9.0x
32,768	8.59e+09	2.58e+10	5.58e+11	65.0x

The costs per layer are $O(nd^2)$ for recurrence, $O(nkd^2)$ for convolution, and $O(n^2d + nd^2)$ for attention.

The crossover is at roughly $n = d$. Below that, attention costs about what recurrence costs and buys a path length of 1. Above it, the quadratic term takes over and every additional token is more expensive than the last.



IN PLAIN ENGLISH

Attention compares every token with every other token. Ten tokens means a hundred comparisons; a thousand tokens means a million; a hundred thousand tokens means ten billion.

That is why context windows were small for years, why they are quoted as a headline feature, and why a substantial slice of current research is devoted to making this cheaper. FlashAttention reduces the memory cost by never storing the full matrix; sparse and sliding-window schemes reintroduce a locality assumption on purpose; linear-attention and state-space approaches change the operation to avoid the square entirely.

Notice what most of those have in common: they buy back efficiency by **putting an assumption back in**. Chapter 21 draws the moral.

20.2 What is given up

Locality. An untrained attention layer has no preference for nearby tokens at all. For language that is mostly a benefit. For images it is a real loss, and it is measured in Chapter 21.

Order. Chapter 19: permutation equivariant, so position must be supplied separately.

Data efficiency. This follows from Chapter 2 with no further argument. A weaker prior means more of the structure must be learned from examples, so more examples are needed.

◉ DEEPER DETAIL: The quadratic cost is not merely time, it is memory

Worth being precise, because it is the constraint people actually hit.

The attention matrix for one head is $n \times n$. At a sequence length of 32,768 that is over a billion entries per head per layer, and during training every one must be **kept** for the backward pass, as Week 3, Chapter 14 explained for the forward pass generally.

This is why long-context training is expensive in a way that long-context **inference** is not, and it is why FlashAttention mattered so much: it computes attention in tiles and recomputes what it needs during the backward pass rather than storing the full matrix. The mathematics is unchanged and the memory cost drops from quadratic to linear, which is a good illustration of an engineering advance with no modelling content at all.

QUIZ WATCH · WEEK 6

Likely: state the computational cost of attention and where the crossover with recurrence sits. Name the three things attention gives up and why. Explain why the quadratic cost is a memory problem as much as a time problem.

21 The Scaling Bargain

COVERS	The trade between built-in structure and data, and the argument this week has been building towards
TIME	About 18 minutes; the chapter that connects Week 4 to Week 5
IN BRIEF	Weak assumptions plus enormous data beat strong assumptions plus modest data. The reverse is also true, and which regime you are in is the whole question.

Everything in this week can be summarised as one trade, and this chapter states it plainly.

KEY IDEA

The scaling bargain. Structure built into an architecture substitutes for data. So:

- With **little** data, the architecture whose assumptions match reality wins, and wins by a large margin.
- With **enormous** data, the architecture that assumes less can learn the structure from examples, and then wins, because it is not constrained by any assumption that is only approximately true.

Neither half is the general rule. Which half applies to you is decided by how much data you have relative to the complexity of the structure to be learned.

21.1 The Vision Transformer, which is the cleanest evidence

The Vision Transformer took the strong hint. Chop an image into patches, treat the patches as tokens, and apply a transformer with no convolutional structure at all. Locality: gone. Translation equivariance: gone. The model must learn from data that neighbouring patches are related.

The published result is exactly what Chapter 2 predicts. Trained on a mid-sized dataset (ImageNet, about 1.3 million images), the Vision Transformer performs **worse** than a comparable CNN. Trained on a much larger one (JFT-300M, about 300 million images), it performs **better**.

Same architecture, same task, opposite conclusion, and the only variable is the amount of data. That is the bargain, demonstrated at scale by people with the compute to run the experiment properly.

MEASURED IN THE LAB

This week's lab shows the same shape at a scale you can run on a laptop. From Chapter 2's tables:

Training images	CNN (strong, matching assumptions)	MLP (no assumptions)
128	0.945	0.275
512	0.992	0.328
4,096	not run	0.730
16,384	not run	0.948

At 128 images the structured model wins by 67 percentage points. Given 128 times the data, the unstructured model catches up. It is the Vision Transformer result in miniature, with a 12,000-times smaller compute budget.

The bitter lesson, and the correct reading of it

Rich Sutton's 2019 essay "The Bitter Lesson" is the standard citation here and it is very often misread, so it is worth stating carefully.

Sutton's observation: across seventy years of AI, methods that leverage more computation have repeatedly beaten methods that build in human knowledge about the domain. Hand-crafted chess heuristics lost to search. Hand-engineered vision features lost to learned ones. Hand-written grammars lost to statistical models.

The misreading is "domain knowledge is useless, just scale up". That is not what the record shows, and it is falsified by the first half of the bargain above.

The defensible reading is about **where** knowledge should enter. Building in knowledge as a **hard constraint on the solution** tends to lose, because your knowledge is incomplete and the constraint eventually blocks the correct answer. Building in knowledge as a **structural prior about the form of the problem** has an excellent record: convolution is exactly such a prior and it is not going anywhere.

There is also a resource argument that the essay's popular form tends to hide. "Just use more computation" is available to organisations with hundreds of millions of dollars. For everybody else, matching your assumptions to your problem is not a quaint alternative; it is the only lever there is. Both halves of the bargain are true, and which one applies to you depends on which side of that divide you are on.

WHY THIS MATTERS

This is the chapter that makes Week 4 useful rather than merely informative, and it is the direct bridge into Week 5.

Week 5 asks the natural next question. If locality is worth 128 times the data because it captures something roughly true about images, what is a **conservation law** worth? A physical law is not a soft statistical regularity; it is exactly true, everywhere, always. Building conservation of energy into a model is a strong assumption that is **not** approximately right but **actually** right.

That is the premise of physics-informed and science-grounded machine learning, and the bargain above tells you exactly when it pays: in the data-scarce regime, which is where nearly all scientific and engineering problems live. You will not have 300 million labelled examples of turbine failures. You will have forty, and the laws of thermodynamics.

Week 3's Chapter 14 noted that a hidden layer **learns** a change of coordinates. Week 4 has shown that an architecture can be **told** one. Week 5 asks what happens when what you tell it is a law of nature.

COMMON MISTAKE

"Transformers replaced CNNs, so convolution is obsolete."

Convolutional networks remain the sensible default when data is limited, when inputs are large relative to the compute available, and when the task genuinely is local. They are used throughout medical imaging, industrial inspection and on-device vision, all of which are data-scarce or compute-constrained.

It is also worth noting that the two families have been converging. Modern vision transformers reintroduce locality through windowed attention (Swin) or hierarchical patching, and modern convolutional networks (ConvNeXt) adopt transformer training recipes and match transformer accuracy. The clean rivalry of 2021 has become a design space with locality as one dial among several, and the dial is set by how much data you have.

QUIZ WATCH · WEEK 6

Very likely: state the scaling bargain in both directions. Explain the Vision Transformer result and why it supports rather than contradicts the week's thesis. Give the defensible reading of the bitter lesson and one criticism of its popular form. Explain what this implies for a project with four thousand training examples.

22 The Same Three Families in PyTorch

COVERS	What the framework replaced, line for line against the numpy you wrote
TIME	About 12 minutes
IN BRIEF	Week 3 wrote backpropagation and then met <code>loss.backward()</code> . This chapter does the same for the three architectures of Week 4.

Week 3, Chapter 16 made an argument worth repeating: a framework is worth understanding **after** you have written the thing by hand, because then you know exactly which lines it replaced.

22.1 The three families, in PyTorch

```
import torch.nn as nn
```

```
# A convolutional network. Chapters 6 to 9.
```

```
cnn = nn.Sequential(
    nn.Conv2d(1, 8, kernel_size=3, padding=1),    # your Conv2D class
    nn.ReLU(),
    nn.Conv2d(8, 8, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.AdaptiveAvgPool2d(1),                    # your GlobalAvgPool
    nn.Flatten(),
    nn.Linear(8, 4),
)
```

```
# A recurrent network. Chapters 13 to 15.
```

```
lstm = nn.LSTM(input_size=5, hidden_size=24, batch_first=True)
```

```
# Attention. Chapters 17 to 19.
```

```
attn = nn.MultiheadAttention(embed_dim=32, num_heads=4, batch_first=True)
```

Three lines of specification for three families that took this booklet sixteen chapters and the lab about seven hundred lines of numpy.

Your numpy	What it did	PyTorch
<code>im2col</code> and the <code>matmul</code> in <code>Conv2D.forward</code>	rearranged patches so the layer is one matrix multiply	<code>nn.Conv2d</code>
the summation in <code>Conv2D.backward</code>	accumulated the gradient across every position, which is weight sharing	<code>loss.backward()</code>
<code>col2im</code> with its accumulation	added up the gradient con-	<code>loss.backward()</code>

	tributions to each pixel	
the for <code>t in range(T)</code> loop in <code>VanillaRNN.forward</code>	applied the same matrix at each step	<code>nn.RNN</code>
the four gate computations and the cell update	the LSTM's additive memory path	<code>nn.LSTM</code>
<code>softmax(Q @ K.T / sqrt(d)) @ V</code>	scaled dot-product attention	<code>F.scaled_dot_product_attention</code>
the softmax Jacobian trick in backward	differentiated the softmax without building the Jacobian	<code>loss.backward()</code>
<code>AdamMixin.step</code>	per-parameter step rescaling	<code>torch.optim.Adam</code>

KEY IDEA

Every entry in the right-hand column replaced **code**, not **understanding**. The assumptions of Chapters 7, 12 and 17 are still there, still doing all the work, and still your responsibility to choose correctly.

`nn.Conv2d` does not tell you whether locality is true of your data. It assumes it, silently and efficiently, and it will assume it just as efficiently when it is false.

✓ IN PRACTICE: Two things the framework will not warn you about

One: the head that discards your assumption. `nn.Flatten()` followed by `nn.Linear` compiles and trains perfectly, and produces the 0.230 from Chapter 9. Nothing errors. There is no warning. Use `nn.AdaptiveAvgPool2d(1)` when you want position-invariance, and know which one you have chosen and why.

Two: shape conventions. PyTorch convolutions expect (batch, channels, height, width). Recurrent layers default to (sequence, batch, features) unless you pass `batch_first=True`, which is a genuinely notorious source of bugs, because feeding the wrong arrangement often does not error: it trains, badly, on nonsense. Week 3's trap with `zero_grad()` was the same species of problem, and the same discipline applies: check your shapes with an assertion, once, at the top.

TRY IT YOURSELF

Run `torch_models.py` in the lab if you have PyTorch installed. It builds all three families, prints parameter counts beside the numpy versions, and confirms they match. If PyTorch is not installed the file says so politely and everything else in the lab still runs.

QUIZ WATCH · WEEK 6

Likely (LO4 is examined): name the PyTorch layer corresponding to each family. State what `loss.backward()` replaced in your convolution code. Explain one trap the framework will not warn you about.

23 Choosing an Architecture

COVERS

A procedure, not a preference: how to decide, and how to defend the decision

TIME

About 14 minutes, then use it for the rest of the module

IN BRIEF

Match the assumption to the data. Three questions decide most cases, and the answers are defensible in a sentence each.

23.1 The decision table

If your data has	Start with	Be- cau
a spatial grid, local patterns, patterns that recur anywhere	convolution	lo- cal- ity and sta- tion- ar- ity are both true, so you get them for free
an order, moderate length, streaming or online arrival	recurrence or a state-space model	fixed cost per step and no need to hold the whole se- quence
relations between distant elements, and plenty of data	attention	path length 1 at any dis-

		tance, and con- tent de- cides rel- evance
no exploitable structure (tabular, engineered features)	gradient-boosted trees, then a dense network	noth- ing to ex- ploit, and trees han- dle mixed types and miss- ing val- ues bet- ter
a graph	a graph neural network	share weights across nodes; the con- nec- tions are given, not as- sumed
a set with no order	attention without positional encoding	per- mu- ta- tion equi- vari- ance is now a fea- ture ,

		per Chap- ter 19
an exactly known physical law	see Week 5	a law is a stronger and more re- li- able prior than any sta- tis- ti- cal reg- ular- ity

That last row is not a placeholder. It is where the module goes next, and Chapter 21 explains why it belongs in this table.

23.2 The three questions

- 1. What must be true of my data for this architecture's assumptions to hold?** State it as a sentence about the world. If you cannot, you do not yet understand the architecture.
- 2. Is that sentence actually true of my data?** This is checkable, sometimes cheaply. Would shuffling the positions destroy the meaning for a human? Does the same pattern mean the same thing everywhere?
- 3. How much data do I have, relative to how much structure must be learned?** Chapter 21's bargain. Scarce data means lean on structure; abundant data permits weaker priors.

✓ IN PRACTICE: Always fit the simple baseline first

Week 3's field guide asked whether a simpler model would have done, and noted that in the Week 3 lab an SVM from the 1990s beat the neural network.

The same discipline applies with more force here, because architectures are more expensive to get wrong. Before building a transformer, fit the convolutional network. Before either, on tabular data, fit gradient-boosted trees. You need to know what the complicated thing **cost** you, and a number you can compare it against.

This week's lab supplies an uncomfortable example: on pre-centred digits the plain dense network (0.978) beat the convolutional one (0.940). Had you assumed the CNN was better because CNNs are for images, you would have shipped the worse model and never known.

TRY IT YOURSELF

Take any dataset you have met, in this module or elsewhere, and answer the three questions in under 150 words. Then name the architecture you would start with and one you would use as a baseline.

By Week 6 this is quiz revision. By Week 12 it is a marked section of your project's critical brief, and "it is what the tutorial used" will not earn the marks.

QUIZ WATCH · WEEK 6

Very likely as an applied question: given a described dataset, choose an architecture and justify it by naming the assumption and arguing that it holds. Expect at least one case where the intuitive answer is wrong, such as tabular data described in a way that makes a CNN sound plausible.

24 Structure, Assumption and Responsibility

COVERS	Week 2's argument, arriving through the architecture
TIME	About 12 minutes
IN BRIEF	An architectural assumption is a claim about the world made on your behalf. If you did not choose it, nobody did, and it is still in your system.

Week 2 argued that a loss function is a values statement. Week 3 extended it: a hyperparameter you did not choose is a decision made by nobody, with your name on the commit. Week 4 completes the sequence, and the architectural version is the least visible of the three.

24.1 The assumption you did not know you made

When you type `nn.Conv2d`, you assert that in your data, nearby things are related and the same pattern means the same thing everywhere. That is a factual claim about the world. It is often true. When it is false, the consequences are not evenly distributed.

▮ CASE FILE: Stationarity as a fairness assumption

Consider a model that assesses skin lesions from photographs, using a convolutional network. Stationarity says a lesion looks the same wherever it appears in the frame, which is reasonable. But consider what the **learned** filters are tuned to: texture and colour contrast against surrounding skin, on the distribution of skin tones in the training data.

If that distribution is overwhelmingly light-skinned, the filters learned in the early layers are tuned to contrast patterns that behave differently on darker skin. The architecture is not at fault in itself. But it **concentrates** the problem: because weights are shared across all positions and all images, a feature detector that works poorly for one group works poorly for that group everywhere, uniformly and invisibly.

This is a documented pattern in dermatology datasets, and the general shape is Week 2, Chapter 4's representational bias meeting Week 4's weight sharing. The architectural choice did not create the disparity. It ensured the disparity was applied consistently, at every position, with high confidence.

WHY THIS MATTERS

The point is not that convolution is ethically suspect. It is that **every** structural choice narrows what the model can express, and what it can no longer express is not a neutral loss.

The texture-over-shape finding in Chapter 11 is the same argument in a technical register: the architecture made texture the path of least resistance, so the model took a shortcut that worked on the test set and failed outside it. Week 2 called that a validity problem. Week 4 gives you its architectural cause, and therefore a place to intervene.

This is also why “we used a standard architecture” is not a defence. Standard means widely used, not appropriate to your problem. The architecture is part of the system you are accountable for.

✓ IN PRACTICE: Three questions to add to a model card

When you document a model, whether for the Week 12 project or for a real deployment, state:

1. **What does this architecture assume about the input?** One sentence, in plain language, no jargon.
2. **For which inputs is that assumption weakest?** Which cases sit furthest from what the structure expects.

3. What did you check? The shuffle test, the shift test, a subgroup breakdown. Any evidence that the assumption holds for the data you will actually see.

A model card that lists the architecture but not its assumptions has documented the implementation and hidden the reasoning, which is the wrong way round.

QUIZ WATCH · WEEK 6

Likely as a discussion prompt rather than a short answer: explain how an architectural assumption can have distributional consequences, and what you would document about it. Connect the texture-over-shape finding to Week 2's account of shortcut learning.

25 A Field Guide: Seven Questions for Any Architecture

COVERS The week, operationalised

TIME About 10 minutes, then use it forever

IN BRIEF Week 1 gave seven questions for a headline, Week 2 for anything you ship, Week 3 for anything you train. These are for anything you design.

1. **What is connected to what, in one layer?** Which inputs can influence which outputs directly. Chapter 1. If you cannot draw it, you do not know the architecture.
2. **Which weights are shared, and across what?** Space, time, or nothing. Chapter 4. This tells you the parameter count, the data efficiency, and whether variable-size input is accepted.
3. **What does that combination assume, as a sentence about the world?** Chapters 7, 12, 17. If you cannot state it without jargon, you cannot check it.
4. **Is that sentence true of my data, and how would I know?** The shuffle test costs one line and settles the locality question. The shift test settles the equivariance question. Chapter 11.
5. **Does my architecture carry the assumption all the way to the output?** Chapter 9. Identical convolutions with a flatten head scored 0.230 where global pooling scored 0.998. The trunk is not the whole model.
6. **What is the cost, and where is the crossover?** Chapter 20. Quadratic in sequence length is fine at 512 tokens and ruinous at 32,768. Know which regime you are in before choosing.
7. **How much data do I have, relative to the structure to be learned?** Chapter 21. Scarce data means lean on structure. This question decides more than the other six combined.

TRY IT YOURSELF

Standing exercise for the rest of the module: answer all seven for any architecture you use, in under 250 words, **before** reporting a result. By Week 6 it is quiz revision. By Week 12 it is a marked section of your critical brief.

26 The Revision Core: The Whole Week on Two Pages

If you memorise nothing else for Week 6, memorise this chapter.

26.1 The one sentence

An architecture encodes assumptions about its data. When those assumptions match reality, models need less data and generalise better. When they do not, structure is dead weight or worse.

26.2 The three families and their assumptions

Family	Assumes	Fails when	Cost per layer
Convolution	locality, stationarity, compositionality: meaning is local, the same pattern matters everywhere, complex things are built from simple ones	positions are arbitrary; position changes meaning; relations are long-range; the transformation is not translation	$O(nkd^2)$
Recurrence	order matters; the same update rule applies at every step; the past fits in a fixed-size summary	the summary is too small (the bottleneck); the gradient is a power of one matrix; steps cannot be parallelised	$O(nd^2)$
Attention	any pair might matter, and content decides which; nothing about order or distance	data is scarce; sequences are very long (quadratic cost); order matters and you forgot the encoding	$O(n^2d + nd^2)$

26.3 The numbers to know

Claim	Number
Dense against convolutional parameters, 224×224 input	6,422,656 against 320, or 20,071 to 1
Convolution's translation equivariance error	exactly 0.00e+00
CNN against MLP, 128 training images	0.945 against 0.275
Data the MLP needed to match the CNN's 128-image score	16,384 images, about 128 times as many
Positions never seen: global pooling against flatten head	0.998 against 0.230, identical convolutions
Pixel shuffle: CNN before and after	0.992 then 0.255, with the MLP unmoved at 0.328 then 0.333
Where the assumption does not pay: pre-centred digits	CNN 0.940, MLP 0.978
RNN gradient reaching step 1 at 90 and 160 steps	7.81e-18 and 1.59e-127
Memory task at 90 steps, RNN against LSTM	0.463 against 1.000
Memory task at 160 steps	both fail: a surviving gradient is necessary, not sufficient
Lookup task, attention across lengths 4 to 25	0.995, 1.000, 0.998, 0.995: flat
Lookup task, LSTM across the same lengths	0.550, 0.362, 0.340, 0.278: decays to chance

Attention's permutation difference without positional encoding	8.88e-16, zero to machine precision
Attention cost against RNN at 32,768 tokens	65 times
Receptive field after eight 3×3 layers with two pools	32×32 , the whole image

26.4 Convolution

Slide a small kernel over the input, taking a weighted sum, **with the same weights at every position**. Size arithmetic: $\text{out} = \left\lfloor \frac{n+2p-k}{s} \right\rfloor + 1$. Parameters: $C_{\text{out}}(C_{\text{in}}k^2) + C_{\text{out}}$, **independent of image size**. A 3×3 filter in a 64-channel layer has 576 weights, not 9. Equivariant to translation, **not invariant**: pooling adds local invariance, global average pooling adds real invariance and restores size-independence. Receptive field: $r_L = r_{L-1} + (k_L - 1) \prod s_i$. First-layer filters learn edges even with no labels, because edges are what images are made of.

26.5 Recurrence

$h_t = \tanh(x_t W_x + h_{t-1} W_h + b)$, with **one** W_h used at every step: weight sharing across time. An unrolled diagram is one layer applied many times, not many layers. The gradient to step 1 is a product of T factors all containing the same W_h , so it behaves like W_h^T : slightly small vanishes, slightly large explodes, and the band between is a knife edge. Clipping fixes explosion only. Adam moves where vanishing bites but does not remove it. Two separate failures: the **vanishing gradient** (a training problem, gates help) and the **bottleneck** (a capacity problem, gates do not help).

26.6 LSTM

Three gates, all sigmoids in $(0, 1)$: forget (keep how much of the old), input (add how much of the new), output (reveal how much). $c_t = f_t c_{t-1} + i_t g_t$ is **additive**, so the gradient along the cell is multiplied only by f_t , which the network chooses. Memory half-life is $\log(0.5)/\log(f)$: $f = 0.95$ gives about 14 steps, $f = 0.999$ gives about 693. Forget-gate bias is initialised positive; at 1.5 this lab's LSTM lost to a plain RNN, at 3.0 it scored 1.000. Same structural idea as ResNet's skip connection: an additive route the gradient is not repeatedly shrunk along.

26.7 Attention

$\text{softmax}(QK^T/\sqrt{d})V$. Four stages: project into query, key and value; score every pair by dot product; normalise with softmax so each row sums to 1; mix the values by those weights. A soft dictionary lookup. Three projections because looking-for, offering, and contributing are different roles. Divide by $\sqrt{d}$ because dot products grow like $\sqrt{d}$ and a saturated softmax has no gradient. **Permutation equivariant**, so positional encoding is compulsory, and sinusoids are used because a counter would grow without bound. Multi-head because one head gives one weighted average and language needs several relations at once. A transformer block is attention (mixes **between** positions) plus a feed-forward network (transforms **within** a position) plus residuals and layer norm; the feed-forward part holds about two thirds of the parameters. Causal masking makes generation possible.

26.8 The bargain

Structure substitutes for data. Scarce data: matching assumptions win, by a lot. Enormous data: weaker assumptions win, because they are not limited by an approximation. The Vision Transformer loses to a CNN on 1.3 million images and beats it on 300 million, same architecture. The bitter lesson, read properly, is about **where** knowledge enters: as a hard constraint on the answer it tends to lose, as a structural prior about the form of the problem it has an excellent record.

26.9 Ten one-line lessons

1. Structure is an assumption about the world, and it is either true or false about your data.
2. A more expressive model can be a worse model, because expressiveness is not a reason to prefer the right answer.
3. Weight sharing is one idea: convolution shares across space, recurrence across time.
4. A shared weight's gradient is the **sum** over every place it was used, which is why CNNs extract more signal per image.
5. Equivariance is not invariance, and a flatten-and-dense head throws the difference away.
6. Depth in time is worse than depth in layers, because it is a power of one matrix rather than a product of different ones.
7. Gates are learned control over how fast the past is forgotten.
8. A surviving gradient is necessary but not sufficient.
9. Attention buys path length 1 at any distance and pays quadratic compute.
10. Weak assumptions need data; if you do not have data, you need true assumptions.

QUIZ WATCH · WEEK 6

The Week 6 quiz draws its Week 4 items from exactly this chapter: the three families and their assumptions, the size and parameter arithmetic, equivariance against invariance, the receptive field, why the RNN gradient is a power of one matrix, what gates do and do not fix, the four stages of attention, permutation equivariance and positional encoding, and the scaling bargain. If you can reconstruct this chapter onto two blank pages from memory, Week 4 revision is done.

27 Glossary in Plain English

Term	Plain-English meaning	Ch.
Inductive bias	Everything a model believes before it sees any data. Compulsory, not a flaw.	2
Locality	The assumption that meaning is decided by what is nearby.	7
Stationarity	The assumption that the same pattern matters equally everywhere.	7
Compositionality	The assumption that complex things are built from simpler things.	7
Weight sharing	Using the same numbers in more than one place, on purpose.	4
Convolution	Slide a small grid of weights over the input, taking a weighted sum at every position.	6
Kernel / filter	The small grid of weights. Nine numbers for a 3×3 on one channel.	6
Feature map	A grid saying how strongly one pattern appears at each position.	8
Channel	One feature map in the stack that a layer outputs.	8
Stride	How far the kernel jumps between positions. Stride 2 halves the output.	6
Padding	A border of zeros added so the kernel can reach the edges.	6
Receptive field	How much of the original input one unit can possibly depend on.	9
Pooling	Keep the largest (or mean) value in each small window; discard the rest.	9
Equivariance	Move the input and the output moves correspondingly. Convolution has this.	7, 9
Invariance	Move the input and the output does not change. Global pooling gives this.	9
Global average pooling	Average each channel over all positions. Discards where, keeps whether.	9
Residual / skip connection	Add the input to the output, so a layer learns a correction.	10
Batch normalisation	Rescale activations using batch statistics. Works reliably; why is contested.	10
Recurrence	Apply the same update at every step, carrying a running summary.	13
Hidden state	The fixed-size running summary an RNN carries forward.	13
Backpropagation through time	The chain rule applied along the unrolled sequence.	14
Vanishing gradient (in time)	The gradient to early steps shrinks as a power of one matrix.	14
Gradient clipping	Cap the gradient's length, keep its direction. Fixes explosion only.	14
Gate	A learned number between 0 and 1 controlling how much passes through.	15
Forget gate	How much of the existing memory to keep at this step.	15
Cell state	The LSTM's memory, updated additively so the gradient can survive.	15

Memory half-life	$\log(0.5)/\log(f)$. How many steps until half the signal is gone.	15
GRU	A two-gate simplification of the LSTM. Cheaper; usually similar.	15
Bottleneck	A fixed-size state cannot hold unboundedly much. Gates do not fix this.	16
Attention	Score every pair, softmax the scores, average the values by those weights.	17, 18
Query, key, value	What I am looking for, what I offer as a match, what I contribute.	18
Scaled dot product	Divide scores by $\sqrt{d}$ so softmax does not saturate as width grows.	18
Multi-head attention	Split the width into slices; each learns its own notion of relevance.	19
Permutation equivariance	Shuffle the input, the output shuffles identically. Attention has this.	19
Positional encoding	Position information added to embeddings, because attention cannot see order.	19
Causal mask	Block attention to later positions, so next-token prediction is meaningful.	19
Transformer block	Attention (between positions) plus feed-forward (within a position), with residuals.	19
Quadratic cost	Attention compares every pair, so cost grows with the square of the length.	20
Scaling bargain	Structure substitutes for data. Scarce data favours structure; abundant data favours weak priors.	21
Transfer learning	Reuse early layers trained elsewhere; retrain only the last few.	8
Adam	Rescale each parameter's step by that parameter's own typical gradient size.	22

28 Question Bank: Practising for Week 6

Answers are not printed. Every question is answerable from a named chapter, and hunting the answer is the revision.

28.1 Short answer: two to three sentences each

1. State in one sentence what an architecture encodes. (Ch. 1)
2. Explain why a more expressive model can perform worse than a less expressive one on the same data. (Ch. 1, 3)
3. Define inductive bias and explain why a learner without one cannot generalise. (Ch. 2)
4. State the relationship between built-in assumptions and data requirements, in both directions. (Ch. 2)
5. Name three places other than the architecture where assumptions enter a system. (Ch. 2)
6. Explain why the MLP won on pre-centred digits without abandoning the week's thesis. (Ch. 2, 11)
7. Explain what the pixel-shuffle experiment demonstrates and why it is a fair test. (Ch. 3)
8. State the three benefits of weight sharing and the assumption each requires. (Ch. 4)
9. Explain what happens to the gradient of a shared weight, and one consequence for data efficiency. (Ch. 4)
10. Explain why an unrolled RNN diagram is not a deep network in the ordinary sense. (Ch. 4, 13)
11. Give the convolution size formula and explain each symbol. (Ch. 5)
12. Explain what a kernel whose weights sum to zero responds to, and why. (Ch. 6)
13. Distinguish padding from stride, and give the purpose of each. (Ch. 6)
14. Explain why the unflipped operation is still called a convolution, and why it does not matter. (Ch. 6)
15. Name the three assumptions of a convolution layer and state each in plain language. (Ch. 7)
16. Explain translation equivariance and why it is a property of the operation rather than something learned. (Ch. 7)
17. Explain what a channel is and how it differs from a feature map. (Ch. 8)
18. Explain the feature hierarchy across depth, and why transfer learning follows from it. (Ch. 8)
19. Explain why ReLU after a convolution separates polarity, and why that matters. (Ch. 8)
20. Define the receptive field and state the recurrence for computing it. (Ch. 9)
21. Distinguish equivariance from invariance, with one example of each. (Ch. 9)
22. Explain why a flatten-and-dense head discards translation equivariance. (Ch. 9)
23. Explain why global average pooling makes a CNN size-independent. (Ch. 9)
24. Explain the degradation problem ResNet addressed, and why rising **training** error rules out overfitting. (Ch. 10)
25. State what a residual block computes and explain how it helps the gradient. (Ch. 10)
26. Name two situations in which a CNN's assumptions fail, with a reason for each. (Ch. 11)
27. Explain the texture-over-shape finding and connect it to the locality assumption. (Ch. 11)
28. Name the three ways sequences differ from fixed grids, and what each rules out. (Ch. 12)
29. State the two assumptions a recurrent network makes, and what each one costs. (Ch. 12)
30. Write the RNN update equation and explain every term in words. (Ch. 13)
31. Explain why an RNN's parameter count is independent of sequence length. (Ch. 13)
32. Explain why the gradient through time is a power of one matrix, and why that is worse than a deep feedforward network. (Ch. 14)
33. Explain what gradient clipping fixes and what it does not. (Ch. 14)
34. State one thing a better optimiser does and does not do about the vanishing gradient. (Ch. 14)
35. Name the three LSTM gates and say what each controls. (Ch. 15)
36. Write the cell update and explain why the additive form helps the gradient. (Ch. 15)
37. Explain why the forget-gate bias is initialised positive, and what happens if it is too small. (Ch. 15)

38. Explain the difference between what gates fix and what they do not, using the 160-step result. (Ch. 15)
39. Explain the fixed-size bottleneck and why gates do not address it. (Ch. 16)
40. Explain why an RNN cannot be parallelised and why that mattered more than accuracy. (Ch. 16)
41. Explain attention as a lookup, and say precisely what makes it “soft”. (Ch. 17)
42. Explain why a weaker assumption can still win, referring to both halves of Chapter 2. (Ch. 17, 21)
43. State the attention formula and explain each of its four stages. (Ch. 18)
44. Explain why there are three separate projections and what each represents. (Ch. 18)
45. Explain why the scores are divided by $\sqrt{d}$, referring to softmax saturation. (Ch. 18)
46. Explain why attention weights are not a reliable explanation of a model’s decision. (Ch. 18)
47. Explain why one attention head is insufficient, and what multi-head attention does about it. (Ch. 19)
48. State what permutation equivariance means for attention and give the consequence. (Ch. 19)
49. Explain why positional encoding uses sinusoids rather than writing the position number in. (Ch. 19)
50. Name the components of a transformer block and say what each one does. (Ch. 19)
51. Explain causal masking and why generative models require it. (Ch. 19)
52. State attention’s cost, and say where the crossover with recurrence sits. (Ch. 20)
53. Explain why attention’s quadratic cost is a memory problem as much as a time problem. (Ch. 20)
54. State the scaling bargain in both directions. (Ch. 21)
55. Explain the Vision Transformer result and why it supports rather than undermines the week’s thesis. (Ch. 21)
56. Give the defensible reading of the bitter lesson, and one criticism of its popular form. (Ch. 21)
57. Name the PyTorch layer for each of the three families, and one trap the framework will not warn you about. (Ch. 22)

28.2 Applied: read the numbers, do the arithmetic

1. A Conv2d(64, 128, 5, stride=2, padding=2) layer receives a $56 \times 56 \times 64$ input. Give the output shape and the parameter count. (Ch. 5, 8)
2. A network is eight 3×3 convolutions with 2×2 pooling after every second one. What is the receptive field after layer 6, and can a unit at layer 6 see all of a 20×20 image? (Ch. 9)
3. Two models have identical convolutional layers. On positions seen in training they score 0.993 and 1.000; on positions never seen they score 0.230 and 0.998. Explain the difference and name the architectural cause. (Ch. 9)
4. A CNN scores 0.992 with the original pixel order and 0.255 after a fixed permutation, while an MLP scores 0.328 and 0.333. Explain both rows and say what the experiment proves. (Ch. 3, 11)
5. On a dataset of pre-centred, pre-cropped digits, a CNN scores 0.940 and an MLP 0.978. Explain the result and say what you would change if you wanted the CNN to win. (Ch. 2, 11)
6. An RNN’s gradient at step 1 is $1.59\text{e-}127$ for a 160-step sequence. State the mechanism, and say whether clipping, Adam or an LSTM would be the appropriate response. (Ch. 14, 15)
7. An LSTM’s forget gate settles at 0.90. Roughly how many steps until 1% of a signal remains, and would this LSTM handle a 200-step dependency? (Ch. 15)
8. An LSTM has healthy gradients throughout training and still degrades from 0.550 to 0.278 as a lookup task lengthens from 3 to 24 pairs. Name the failure mode and say why gates cannot fix it. (Ch. 16)
9. A colleague reports that attention weights show their model “is looking at the right words”. Give two reasons to be cautious. (Ch. 18)
10. You have 4,000 labelled images of a manufacturing defect. A colleague proposes a Vision Transformer trained from scratch. Give your recommendation with reasons, and name one alternative that would use their idea safely. (Ch. 21, 23)
11. For $d = 512$, at what sequence length does attention’s quadratic term equal its linear term, and what follows for a 128,000-token context? (Ch. 20)

12. You are given a table of 40 columns of laboratory measurements in arbitrary order. A colleague suggests reshaping each row into a 8×5 grid and applying a CNN. Explain precisely which assumptions this asserts and why each is false. (Ch. 7, 11, 23)

28.3 Discussion prompts

1. Week 2 argued that a loss function is a values statement. Make the equivalent argument for an architecture, and say what you would put in a model card as a result. (Ch. 24)
2. The bitter lesson says built-in human knowledge tends to lose to computation. This week's lab shows a 700-parameter model beating a 66,820-parameter one by 67 percentage points. Reconcile these, then say which matters more for a graduate working in Irish industry.
3. Attention gave up locality and order, which are two true and useful assumptions about many kinds of data, and won anyway. What would have to be true for the same trade to be a mistake?
4. Efficient-attention methods largely work by reintroducing a locality assumption. Is that progress, or a return to convolution with extra steps? Argue both, then commit.
5. The texture-over-shape result shows a model reaching high accuracy by a route its designers did not intend. How would you have detected this before deployment, and what does your answer say about the adequacy of a held-out test set?

29 Further Reading

Optional. Nothing here is examinable.

Under an hour. Andrej Karpathy’s “The Unreasonable Effectiveness of Recurrent Neural Networks” remains the best short introduction to why sequence models were exciting, and it is worth reading now that you know what its models could not do. Christopher Olah’s “Understanding LSTM Networks” is the clearest diagrammatic explanation of Chapter 15 in existence. Jay Alammar’s “The Illustrated Transformer” is the standard visual companion to Chapters 18 and 19, and works well immediately after this booklet.

An evening. Karpathy’s “Let’s build GPT: from scratch, in code, spelled out” builds a working transformer in about two hours of video, and after Chapters 17 to 19 you will recognise every line. The original transformer paper, Vaswani and colleagues (2017), “Attention Is All You Need”, is unusually readable and worth reading for its title’s argument as much as its architecture. Dosovitskiy and colleagues (2021) is the Vision Transformer paper and contains the data-regime result of Chapter 21 in its own tables.

A weekend or more. Goodfellow, Bengio and Courville, **Deep Learning**, Chapters 9 and 10 cover convolution and recurrence properly and are free online. Bronstein, Bruna, Cohen and Veličković, **Geometric Deep Learning**, is the systematic treatment of the idea in Chapter 7’s deeper-detail box: architectures as consequences of symmetries. It is demanding and it is the most intellectually satisfying thing on this list. Geirhos and colleagues (2019) is the texture-over-shape paper. For Chapter 18’s caution about interpretability, read Jain and Wallace (2019) and then Wiegrefe and Pinter’s reply, in that order.

Appendix: How Week 4 Maps to the Module

Module element	Week 4 connection
LO2 (characterise the training of model families)	Chapters 6 to 22; the training loop is unchanged from Week 3 and only the first box differs. Assessed in the Week 6 quiz.
LO3 (compare and contrast architectures and their assumptions)	The whole booklet, and Chapters 1, 7, 12, 17, 21 and 23 in particular. Assessed in the Week 6 quiz.
Indicative content 4, 5	Convolutional and recurrent architectures; attention and transformer models.
Week 1 (history)	The 1989 to 2012 to 2017 lineage of Chapter 10, and Chapter 16's account of how quickly recurrence was displaced once the hardware argument was made.
Week 2 (responsible AI)	Chapter 24: an architectural assumption is a claim about the world made on your behalf. The texture-over-shape shortcut of Chapter 11 is Week 2's validity problem with an architectural cause.
Week 3 (how machines learn)	The loop is identical; only the first box changes. Week 3's vanishing gradient returns in Chapter 14 in a worse form; its hidden-layer change of coordinates becomes this week's central theme; its "fit the simple baseline" discipline returns in Chapter 23.
Week 5 (science-grounded ML)	Chapter 21 is the bridge. A hidden layer learns a change of coordinates; a convolution is told one; a physics-informed network is told a law of nature. The scaling bargain says exactly when that pays: in the data-scarce regime, which is where science lives.
Weeks 7 to 9 (LLMs, scale-free, lean)	Chapters 17 to 21 are the prerequisite. Week 7's models are Chapter 19's block repeated; Chapter 20's quadratic cost is why efficiency research exists.
Week 12 (project)	Chapter 25's seven questions are the architecture section of your critical brief, and Chapter 23's three questions are how you defend the choice.
Convolution / Feature-Map Visualiser	Chapters 6 to 9 as a manipulable object: walk the kernel, grow the receptive field,

	and break the locality assumption on purpose.
Attention Visualiser	Chapters 17 to 20: real dot products on real embeddings, the four stages for one query, and the permutation test that justifies positional encoding.
Sequence Memory Explorer	Chapters 13 to 16: the gradient through time measured live, the forget gate's half-life, and weight sharing across time.

End of the Week 4 Reference Booklet. Next: Week 5, science-grounded machine learning, where the assumption you build in stops being a statistical hunch and becomes a law of nature.